# EventHub Project Report

React Frontend, Django Backend, and Express Comparison

Formal Technical Report with Attached Appendix

**Alla JAMMINE**
Professor

**Mehdi AGHAEI**
Frontend developer

**Maksym DOLHOV**
Backend developer

Universite Paris Cite

April 9, 2026

| Frontend | Primary Backend | Comparison | Deliverables |
|---|---|---|---|
| • React SPA<br>• Responsive CSS<br>• User workflows | • Django<br>• REST API<br>• Validation | • Node.js<br>• Express<br>• Trade-offs | • Formal report<br>• Attached appendix |

EventHub is a full-stack course project about event management. We built a responsive React SPA, used Django for the main backend, and kept a smaller Node.js / Express backend for comparison. The frontend includes a public home page at /, a retro 1998-inspired style, dashboard navigation, and modal CRUD flows.

**Attention !!!**
The **first 7 pages** of this document are the formal report for the project, and the appendix after that contains extra explanations, examples, and notes.

# 1 Introduction

EventHub is a full-stack web application for managing events, participants, and registrations. The project combines a React frontend, a main backend in **Django**, and a smaller comparison backend in **Node.js and Express**. The wweb app opens on a public landing page, lets users log in or register through a modal, and gives authenticated users access to the dashboard, event pages, participant pages, and registration tools. This report explains both how the project was built and why these technical choices were made.

The frontend handles interaction, layout, navigation, rendering, and browser storage. The backend handles data models, validation, routing, and JSON responses. The two sides communicate through HTTP requests and REST-style API endpoints.

| Team member | Main role |
|---|---|
| Mehdi AGHAEI | Frontend developer responsible for the React interface, responsive HTML/CSS structure, and user-facing interaction flow |
| Maksym DOLHOV | Backend developer responsible for the Django API, backend validation logic, and the Node.js / Express comparison backend |

# 2 Project Scope and Requirements

The application needs clear screens for events, participants, and the registration relation that links them. It also needs to work on desktop and mobile screens, so responsive design is not just a visual extra. In the last stage of the project, we also cleaned up the frontend: we redesigned the landing page in a retro 1998 style, removed draft text, simplified navigation, and changed the create or edit flows into smaller pop-up windows that fit the rest of the interface better.

| Requirement | Technical response in EventHub |
|---|---|
| Responsive SPA | React frontend with component-based rendering, responsive CSS layouts, and a collapsible authenticated sidebar on small screens |
| Main backend in Django | Relational models, serializers, viewsets, validation rules, and REST endpoints |
| Backend comparison | Separate Node.js and Express service kept smaller and used as a comparative study backend |
| Event registration logic | Dedicated `Registration` model instead of a hidden many-to-many shortcut |
| Team collaboration | One repository with separate folders for frontend, Django backend, Node backend, and report material |

# 3 System Architecture

The architecture follows a clear full-stack separation. The browser loads the React SPA, the SPA sends requests to the Django REST API for the main application logic, the database stores persistent data, and a smaller Node.js and Express backend is kept in the repository as a comparative path. In the final local setup, Django runs on `localhost:8000`, Vite serves the React frontend on its development port, and Express runs independently on `localhost:3001`. This separation keeps responsibilities clear and allows frontend and backend work to progress in parallel.

| Layer | Responsibility |
|---|---|
| Browser | Hosts the visible user interface and executes frontend JavaScript |
| React SPA | Renders pages, manages state, sends API requests, and updates the UI without full reloads |
| Django REST API | Applies the main business rules, validates data, and exposes the official CRUD endpoints |
| Database | Persists events, participants, and registrations safely across sessions |
| Node.js / Express | Provides a smaller comparison backend for `/api/events`, `/api/participants`, and `/api/health` |

In EventHub, the main application flow goes through Django, while Node.js and Express are kept to show a lighter and more manual backend style. The Express backend does not implement the full authentication or registration workflow; those routes stay on the Django side.

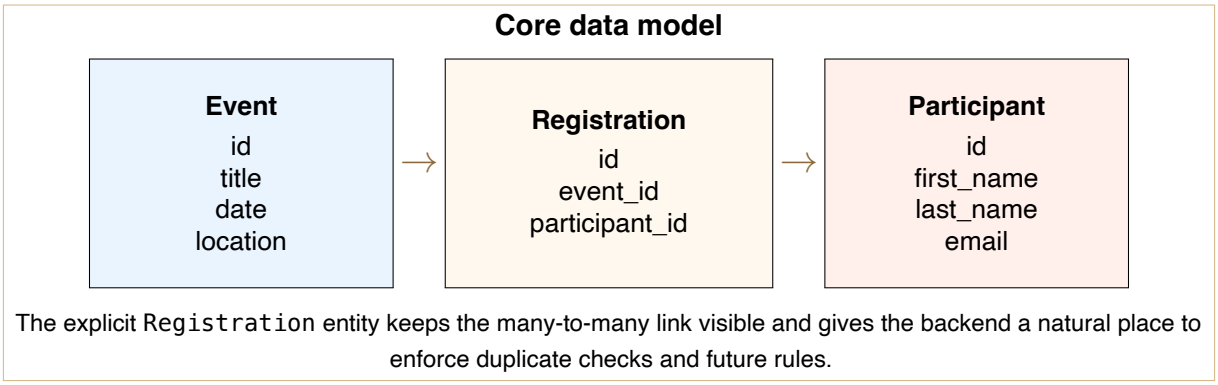
# 4 Backend Design

## 4.1 Django as the primary backend

Django is the main backend because the project is centered on structured relational data and on business rules that should be enforced consistently. Events, participants, and registrations form a classic business-domain model. Django is well suited for this because it provides models, migrations, validation, an administrative interface, and a predictable project structure.

| Django element | Project role |
|---|---|
| Models | Define the database structure for events, participants, and registrations |
| Migrations | Apply schema changes safely to the database |
| Serializers | Convert model objects to JSON and validate incoming request payloads |
| ViewSets | Expose CRUD behavior with a compact REST-oriented structure |
| Router and URLs | Connect endpoints such as `/api/events/` to the correct logic |
| Admin | Provide a fast inspection and management interface during development and testing |

One important modeling decision is the explicit `Registration` model. It is not just a link between two tables. It is also the place where registration logic can live. For example, it can prevent duplicate registrations, store registration-specific data, and later support richer states such as approval or payment status.



| Entity | Purpose |
|---|---|
| Event | Stores the title, date, location, and scheduling information for one event |
| Participant | Stores the identity data of a person who can register for events |
| Registration | Stores the explicit link between one participant and one event, which makes validation and reporting easier |

## 4.2 API organization and validation

The API must expose the core resources of the application in a consistent way. The frontend should not depend on custom ad hoc endpoints when standard resource-based endpoints are sufficient. A compact REST structure therefore remains the clearest choice for the main project.

| Method | Endpoint | Role |
|---|---|---|
| GET | `/api/events/` | Retrieve the list of events for the frontend |
| POST | `/api/events/` | Create a new event from validated form data |
| GET | `/api/participants/` | Retrieve participants and support administration tasks |
| POST | `/api/registrations/` | Create a new event registration after validation |

Validation is a major backend responsibility. The frontend may offer helpful messages, but only the backend can be trusted as the final decision point. In EventHub this means the backend must reject invalid or duplicate registrations, enforce unique participant information where necessary, and return clear error messages when a rule is violated. This approach keeps the data trustworthy even if the frontend is bypassed or a malformed request is sent manually.

## 4.3 Node.js and Express as the comparison backend

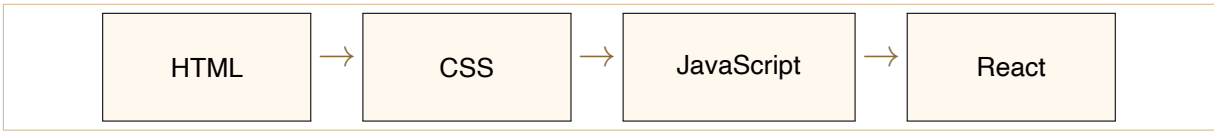
Node.js and Express are kept in the project to make the backend comparison concrete. **Node.js** is the JavaScript runtime. **Express** is the lightweight framework that defines routes and HTTP behavior. In this project, the Node backend is intentionally smaller. Its role is to illustrate a more manual style of backend development rather than to replace the Django service. In simple terms, Django is a full framework with many built-in tools, while Express starts lighter and leaves more decisions to the developer.

The Node workspace lives in `node/`. It exposes a health check plus event and participant resource routes on port `3001`. It uses `express` for routing, `cors` for local frontend access, and `better-sqlite3` for a small local SQLite database stored separately from the Django database. The Node side is organized mainly around `src/server.js`, `src/db.js`, and route files for events and participants. This makes the contrast easier to explain: Django provides a structured project layout, serializers, migrations, and admin support, while Express shows the same kind of API with more manual organization.

| Aspect | Django | Node.js / Express |
|---|---|---|
| Project structure | Enforced project and app layout with migrations | Flexible file layout with manual organization |
| Framework philosophy | Batteries included and opinionated | Minimal and unopinionated |
| Validation and schema | Serializer, model rules, and tracked migrations | Manual checks and SQL changes unless extra tools are added |
| Built-in tools | ORM, admin, auth, and routing conventions | Pick packages and patterns manually |
| Concurrency style | Common Python multi-worker setup | Event-loop model, strong for I/O-heavy work |

This comparison helps justify the final architecture. Django is the better main backend for EventHub because the application depends on models, authentication, validation, and a clear CRUD structure. Express is still useful here, but mainly as a comparison tool that shows what becomes more manual when the framework is lighter.

# 5 Frontend Design




The frontend is a modern React Single Page Application (SPA) that serves as the primary interface for the application. It features a visually engaging public home page, a distinctive retro-styled authentication modal, and a clean, consistent header that enhances navigation and branding. After sign-in, users access personalized, protected content, while compact pop-up forms streamline create and edit actions. The application also includes a theme switcher for user customization, a local visitor counter for engagement, utility links for quick access to support, and a favicon for a polished, professional touch. Each element is designed with UI/UX best practices in mind, ensuring an intuitive, cohesive, and memorable user experience.

## 5.1 HTML, CSS, and React responsibilities

The frontend is built on three complementary layers. HTML provides semantic structure. CSS provides visual layout and responsive behavior. React organizes the page into reusable components and keeps the visible interface synchronized with state changes.

Imagine a React application structured hierarchically as:

- `App`
  - `Header`
  - `MainContent`
    - `Sidebar`
    - `Page`
      - `UserProfile`

Previously, sharing data like `currentUser` from `App` to `UserProfile` required passing it through all intermediate components (`MainContent`, `Sidebar`, `Page`), even when unnecessary. To resolve

this, we implemented a shared `WorkspaceContext` using React's `useContext()` hook, enabling direct access to shared data—such as the current user, role, events, participants, registrations, and navigation helpers—without prop drilling.

| Frontend layer | Design role in EventHub |
|---|---|
| HTML / JSX | Builds semantic structures such as headers, forms, utility bars, cards, pop-up windows, and sections |
| CSS | Controls spacing, retro visual styling, responsive layouts, theme variants, and modal presentation |
| JavaScript | Handles events, filtering logic, API requests, browser storage, routing, and state transitions |
| React | Composes the interface into pages and reusable components with predictable rendering |

## 5.2 Planned screens and user workflows

The frontend design is organized around the main user tasks rather than around isolated widgets. Each page exists to support one clear operation in the project domain.

| Screen or module | Purpose in the final frontend |
|---|---|
| Public landing page | Open at `/`, preview events, surface utility links, and guide users toward sign-in or registration |
| Authentication modal | Provide sign-in, registration, and retro social sign-in buttons without leaving the current page |
| Dashboard | Overview of the application and quick access to major actions from a consistent authenticated shell |
| Events workspace | Browse, filter, create, edit, and inspect events with small pop-up windows for CRUD actions |
| Event detail / registration view | Show one event clearly and allow registration management with responsive forms |
| Participants workspace | Manage participant information and support CRUD operations through compact dialogs |

Responsive design is treated as a functional requirement. The page must remain readable on desktop and mobile widths, which means the CSS strategy must favor flexible grids, meaningful spacing, scalable typography, and component layouts that can collapse from multiple columns to one column without breaking the content hierarchy. In practice this means, for example, that the sidebar collapses behind a toggle on smaller screens, public event cards stay readable without overflow, modal forms reflow into single-column layouts, and the social sign-in buttons stack cleanly when space becomes limited.

### 5.3 Frontend Highlights

| Implemented feature | Why it matters |
|---|---|
| Consistent top-right controls | The theme toggle and auth or workspace actions now stay in the same place across public and authenticated views |
| Pop-up CRUD forms | Create and edit actions for events and participants are easier to understand and fit better on narrow layouts |
| Public event warning modal | Unauthenticated visitors can click public event previews and are clearly prompted to sign in or register for full details |
| React hooks with `useState()` and `useEffect()` | The frontend uses component state for filters, forms, modals, session feedback, and theme changes, and uses effects for API loading, route or theme synchronization, timed banner cleanup, and browser event listeners |
| Workspace context with `useContext()` | Reduces prop drilling by sharing authenticated workspace state and navigation helpers across the React shell and pages |

### 5.4 Frontend-to-backend communication

The frontend communicates with the backend through **JSON**-based **HTTP** requests, following *REST* principles. In practice, each API call involves three common states: *loading*, *success*, and *error*. The *loading* state represents the period during which the request is being sent and the application is waiting for a response. The *success* state indicates that the request has been completed correctly and that a valid response has been received from the server. The *error* state is used when the request fails, for example because of network issues, server problems, or invalid input. Handling these three states is an important part of frontend design, especially in single-page applications, where communication with the backend is asynchronous and cannot always be assumed to work instantly or reliably.

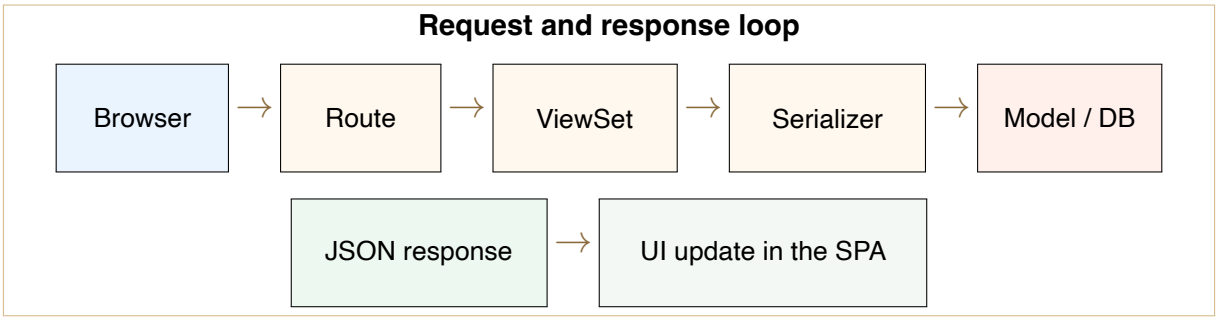
To make the interaction clearer for the user, the interface includes a simple status banner that displays short feedback messages, such as the result of a login or registration attempt. These messages help the user understand what is happening in the application without interrupting the overall experience. When the notification is not essential, it is automatically dismissed after a short period of time in order to keep the interface clean and avoid unnecessary visual clutter.

Form submission follows the same structured logic. After the user enters the required information, the frontend sends a well-organized **JSON** payload to the backend and waits for the asynchronous response. Once the response is received, the interface either updates the displayed content or shows validation errors and feedback messages, depending on the result. This approach highlights a clear separation of concerns: the frontend acts as a disciplined client responsible for presentation and user interaction, while the backend remains responsible for validation, business logic, and data persistence.

## 6 Integration, Repository Structure, and Deployment Plan

Integration is the point where architecture becomes observable. The frontend sends an HTTP request, the backend routes the request, validates the data, interacts with the database, and returns

JSON. The frontend then uses that response to update the visible interface. In the current project, this integration is visible from the public home page all the way to protected CRUD flows, event detail management, and role-aware dashboard behavior.




| Integration concern | Planned handling |
|---|---|
| CORS and local development | Django and Express configured to allow local Vite development requests during integration |
| Data exchange | JSON payloads with stable field names and predictable validation responses |
| Frontend verification | `npm run lint` and `npm run build` confirm that the SPA compiles and remains structurally valid after UI refinements |
| Usability verification | Responsive checks, overflow fixes, modal adjustments, and placeholder cleanup keep the interface clean and usable |
| Node comparison verification | Use Node.js 22, run `npm run dev` in `node/`, then check `http://localhost:3001/api/health` |
| Deployment | Separate production-ready frontend and backend hosting with environment-specific configuration |

## 7 Conclusion

The main technical idea behind EventHub is simple. The project combines a responsive SPA frontend, a structured Django backend, and a smaller Node.js and Express comparison backend, and each part has a clear role. Django is the right main backend choice for this kind of application, React fits the SPA structure well, and the repository layout supports team work and integration.

The report ends here. Thank you for your attention.

The longer appendix continues below in the same PDF and gives extra explanations, worked examples, and revision material connected to the final EventHub implementation.

# Supporting Appendix

This appendix is written as supporting material after the formal report. The idea is to keep the core explanations in one place, with a simple order: **concept -> solved code -> result -> explanation**. When a question appears, the worked solution is given just after it.

## A How to Read This Appendix

This appendix is longer than the formal report because it has three roles. First, it works as a study guide for the main web concepts. Second, it helps with oral defense preparation when technical choices need to be justified quickly. Third, it acts as a revision support for the final EventHub implementation, especially after the landing page, visual style, and frontend behavior were improved in the final version.

| Appendix part | What it is for |
|---|---|
| Frontend learning path | Rebuild the foundations of HTML, CSS, JavaScript, React, and state thinking from simple examples |
| Backend learning path | Revisit API structure, Django models, serializers, routing, and validation rules |
| Live demonstration notes | Prepare a short, credible story for the final project defense |
| Independent study bank | Review theory by topic without opening the original lecture material |
| CC1 practice section | Train on mixed theory and practical questions in the style of the announced exam |

The easiest way to read the appendix is to jump to the part you need. If the question is mainly visual, start with the frontend sections. If it is about validation, data flow, or the relation between events and participants, go to the backend sections. If the question is about the final defense, go directly to the live demonstration storyboard and the follow-up question tables.

### A.1 How the appendix maps to the finished EventHub project

Because the frontend changed a lot during the final phase, it is useful to connect the appendix to the screens that now exist in the repository.

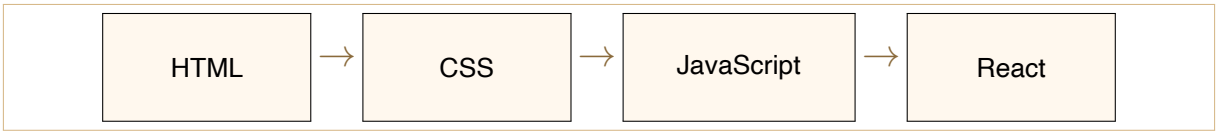
| Final EventHub area | Appendix concept that explains it |
|---|---|
| Public landing page at `/` | HTML structure, CSS layout, responsive thinking, and click-handling logic |
| Authentication modal and social sign-in row | forms, state switching, button semantics, event handling, and small reusable React components |
| Theme toggle, visitor counter, and utility links | browser storage, conditional rendering, and user-interface feedback |
| Dashboard with collapsible sidebar | responsive layout strategy, state-driven navigation, and role-aware rendering |
| Events and participants pop-up forms | controlled inputs, modal composition, validation messages, and CRUD request flow |
| Event detail page and registrations | relation modeling, API communication, backend validation, and status management |

**Checklist.** Read the appendix in this order when revising the final project quickly: first the formal report pages for the big picture, then the appendix introduction above, then the frontend learning path, then the backend sections, then the live demo storyboard, and finally the CC1 practice set.

**Memory hook.** The appendix is not extra decoration. It is the explanation layer of the project. The formal report states what EventHub is and why it was built this way; the appendix teaches how the same ideas work step by step.

# B Frontend Learning Path

## B.1 Basic concepts




| Term | Meaning |
|---|---|
| HTML | Structures the page content: headings, forms, buttons, sections, inputs |
| CSS | Styles the page and makes it responsive |
| JavaScript | Adds behavior such as events, logic, and asynchronous requests |
| React | Organizes the frontend as components and updates the UI when state changes |
| API | The communication contract between frontend and backend, usually through JSON over HTTP |

**Memory hook.** Frontend work is easiest to remember in this order: **HTML = structure**, **CSS = appearance**, **JavaScript = behavior**, **React = state-driven organization**. If one layer feels confusing, ask first which of these four jobs you are actually solving.

## B.2 What each frontend technology does

**HTML** answers the question: *what is on the page?*

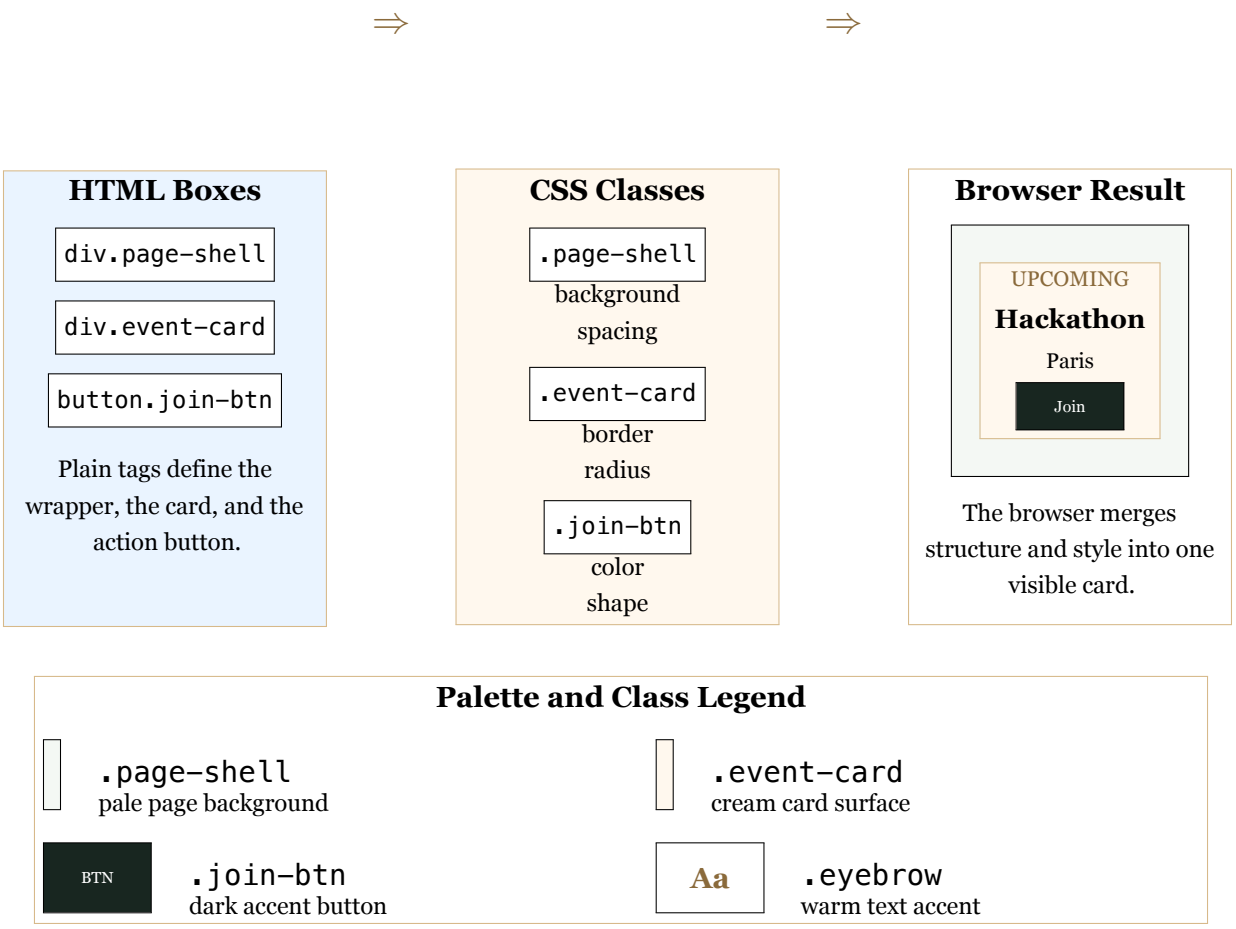
**CSS** answers the question: *what does the page look like on different screens?*

**JavaScript** answers the question: *what happens when the user clicks, types, or requests data?*

**React** answers the question: *how do we keep the interface synchronized with the current application state?*

## B.3 How divs and CSS classes become design

A `div` is only a generic rectangular container. By itself, it does not create a card, a panel, spacing, rounded corners, or color. The visible design appears when a CSS class is attached to the element and the browser applies the rules defined for that class.




Listing 1: HTML structure with div containers and classes

```
<div class="page-shell">
  <div class="event-card">
    <p class="eyebrow">Upcoming event</p>
    <h2>Spring Hackathon</h2>
    <p class="event-meta">Paris - 21 April</p>
    <button class="join-btn">Join</button>
  </div>
</div>
```

Listing 2: CSS classes that turn plain boxes into a design

```
.page-shell {
  min-height: 100vh;
  padding: 2rem;
  background: #f4f8f4;
}

.event-card {
  max-width: 28rem;
  padding: 1.5rem;
  border: 1px solid #d7b98a;
  border-radius: 20px;
  background: #fff8ee;
}

.eyebrow {
  color: #8a6a3b;
  text-transform: uppercase;
```

```
}

.join-btn {
  border: none;
  padding: 0.75rem 1rem;
  border-radius: 999px;
  background: #182620;
  color: white;
}
```

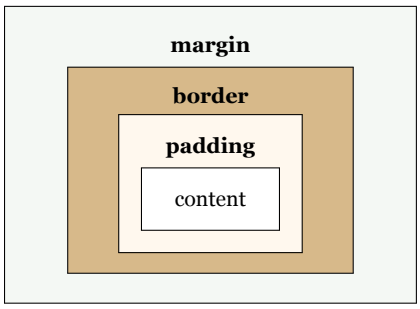
Listing 3: Design result in a browser window

```
+-----------------------------------------------+
| pale green page background                    |
|  +----------------------------------------+   |
|  | cream card                             |   |
|  | UPCOMING EVENT                         |   |
|  | Spring Hackathon                       |   |
|  | Paris - 21 April                       |   |
|  | [ dark green Join button ]             |   |
|  +----------------------------------------+   |
+-----------------------------------------------+
```

The key rule is simple: HTML gives boxes and labels, CSS classes give visual behavior, and the browser combines both to produce the final design that the user sees.

## B.4 Padding, margin, border, and the visible box model

Students often write a `div` and then wonder why the result still looks flat or stuck to the edges. The answer is the box model. Every visible card, alert, button, menu, or section is built from the same four layers: content, padding, border, and margin.




**How to read the layers**

**margin**: outside space between two elements
**border**: the visible edge of the card or button
**padding**: inside breathing room around the content
**content**: the text, image, or button label itself

Listing 4: A simple card that shows padding and margin clearly

```
<section class="layout">
  <div class="note-card">Registration opens at 10:00.</div>
  <div class="note-card">Bring your student ID.</div>
</section>
```

Listing 5: CSS that creates visible spacing around and inside each div

```
.layout {
  background: #edf4ed;
  padding: 24px;
}
```

```
.note-card {
  margin-bottom: 18px;
  padding: 16px;
  border: 2px solid #d7b98a;
  border-radius: 14px;
  background: #fff8ee;
}
```

Listing 6: What the browser shows after applying the classes

```
page area
+-------------------------------------------------+
| layout padding                                  |
|   +-----------------------------------------+   |
|   | note-card padding                       |   |
|   | Registration opens at 10:00.            |   |
|   +-----------------------------------------+   |
|             <- margin-bottom gap ->              |
|   +-----------------------------------------+   |
|   | note-card padding                       |   |
|   | Bring your student ID.                  |   |
|   +-----------------------------------------+   |
+-------------------------------------------------+
```

**Explanation.** Padding creates breathing space inside the card, so text does not touch the border. Margin creates separation outside the card, so two blocks do not collide. When students understand this visual rule, most early CSS layout mistakes become much easier to fix.

## B.5 Mini Active Learning: frontend structure and CSS

**Question 1.** Why does a plain `div` usually look invisible at first?
**Answer.** Because a `div` is only a generic container. It becomes visually meaningful only after CSS adds spacing, borders, colors, size, and layout rules.

**Question 2.** What is the difference between padding and margin?
**Answer.** Padding is inside the element around the content. Margin is outside the element and separates it from neighboring elements.

**Question 3.** If two cards are touching each other vertically, which property is the first one to check?
**Answer.** Usually `margin-bottom` on the first card or `gap` on the parent layout.

## B.6 Step 1: create the frontend entry point

The first job is to have one React entry file and one top-level component.

Listing 7: Minimal React entry point

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
```

```
    </StrictMode>,
  )
```

Listing 8: Expected result

```
The browser mounts the React application inside #root.
```

**Line-by-line explanation.**
`import { StrictMode } from 'react'` loads a helper that warns about unsafe patterns during development.
`createRoot(...)` tells React where to render the application in the HTML page.
`App` is the main component that contains your visible interface.
`<StrictMode><App /></StrictMode>` means: render the App component inside the root container.

## B.7 Step 2: write the first component

The first component can be simple. The purpose is to confirm that the frontend is alive and structured correctly before adding real application features.

Listing 9: Simple React component

```
function App() {
  return (
    <main className="app-shell">
      <section className="hello-card">
        <p className="eyebrow">EventHub starter</p>
        <h1>Hello, World.</h1>
        <p className="message">
          This frontend is the starting point for EventHub.
        </p>
      </section>
    </main>
  )
}

export default App
```

Listing 10: Visible output

```
EventHub starter
Hello, World.
This frontend is the starting point for EventHub.
```

**Why this works.**
`function App()` defines one React component.
`return (...)` tells React what HTML-like structure must appear on screen.
`className` is the React version of the HTML `class` attribute.
The component is small on purpose: first verify that React renders correctly, then grow the page step by step.

## B.8 Step 3: understand the HTML structure

Even inside React, it is important to think in HTML structure first. A good SPA screen still needs a clear semantic layout.

Listing 11: Example semantic structure

```
<main>
  <header>
    <h1>EventHub</h1>
  </header>

  <section>
    <h2>Events</h2>
  </section>

  <section>
    <h2>Participants</h2>
  </section>
</main>
```

This is the right mindset: first define the structure of the page, then style it, then connect it to live data.

## B.9 Step 4: use CSS to make the page responsive

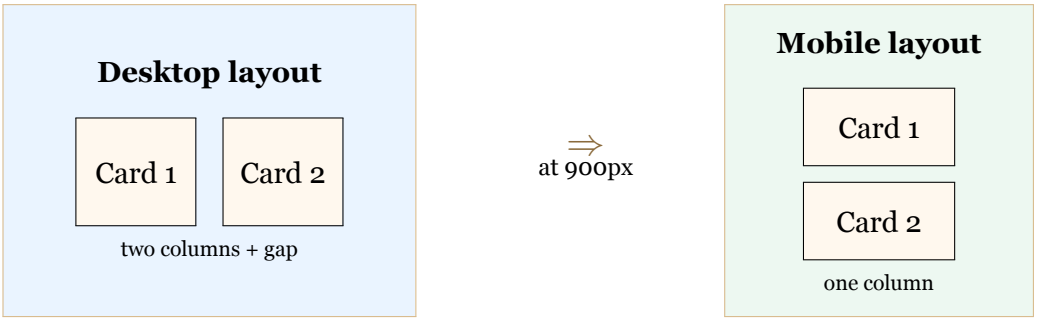
Responsive design means the same screen must stay readable on large and small devices.

Listing 12: Example responsive CSS

```
.content-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 1.5rem;
}

@media (max-width: 900px) {
  .content-grid {
    grid-template-columns: 1fr;
  }
}
```

Listing 13: Expected effect

```
Desktop: two columns
Mobile: one column
```



This drawing shows the same content adapting to screen width without changing the data itself. Good responsive design keeps the content hierarchy stable while the layout becomes simpler and easier to read on smaller screens.

## B.10 Step 5: understand JavaScript in the frontend

JavaScript is what allows the page to react to user actions and backend data.

Listing 14: Basic JavaScript data handling

```
const events = [
  { id: 1, title: 'Hackathon' },
  { id: 2, title: 'Workshop' },
]

for (const event of events) {
  console.log(event.title)
}
```

Listing 15: Console output

```
Hackathon
Workshop
```

## B.11 Step 6: fetch data from an API

At this point the SPA stops being static. It becomes a real application by loading data from the backend.

Listing 16: Basic fetch pattern

```
async function loadEvents() {
  const response = await fetch('http://localhost:8000/api/events/')

  if (!response.ok) {
    throw new Error('Failed to load events')
  }

  return response.json()
}
```

Listing 17: Typical JSON result

```
[
  {
    "id": 1,
    "title": "Spring Hackathon",
    "location": "Paris"
  }
]
```

**How to read this fetch code.**
`await fetch(...)` sends an HTTP request to the backend.
`response.ok` checks whether the server answered successfully.
`throw new Error(...)` stops the normal flow and moves to the error path if the server failed.
`return response.json()` converts the HTTP response body into JavaScript data that React can render.

## B.12 Mini Active Learning: React and API flow

**Question 1.** Why does a SPA need a loading state during `fetch()`?
**Answer.** Because the request is asynchronous and the UI must explain what is happening while waiting for the backend.

**Question 2.** Where does the JSON response come from in this project?
**Answer.** From the backend API, mainly the Django REST API.

**Question 3.** Why is JavaScript still essential even when React is used?
**Answer.** Because React components still rely on JavaScript for arrays, objects, conditions, loops, events, asynchronous code, and state updates.

## B.13 Step 7: move from basics to the actual required pages

Once the basics are understood, the frontend can be built page by page:

1. Login page
2. Dashboard
3. Events list with filtering
4. Event details with registrations
5. Participants page

The correct order is important. Start with static layout, then add local state, then connect the backend, then add route protection and authentication.

## B.14 Step 8: read JSX from outside to inside

Beginners often try to read JSX line by line without first seeing the full component tree. A better method is to read JSX from the outside to the inside. First identify the page shell, then the big sections, then the repeated child components, and only after that read the small props and text details.

Listing 18: A small dashboard read from outside to inside

```
function Dashboard({ events, query, setQuery }) {
  return (
    <main className="dashboard-shell">
      <header className="hero">
        <h1>EventHub</h1>
        <p>Manage events and participants.</p>
      </header>
```

```
      <section className="toolbar">
        <input
          value={query}
          onChange={(event) => setQuery(event.target.value)}
          placeholder="Search events"
        />
      </section>

      <section className="event-grid">
        {events.map((event) => (
          <EventCard
            key={event.id}
            title={event.title}
            location={event.location}
          />
        ))}
      </section>
    </main>
  )
}
```

Listing 19: Mental reading order

```
1. main.dashboard-shell  -> the whole page
2. header.hero           -> the top introduction area
3. section.toolbar       -> the controls area
4. section.event-grid    -> the repeated cards area
5. EventCard props       -> the data each card receives
```

**How to read the code above.**
`function Dashboard(...)` defines one component that receives data and handlers from a parent.
`<main className="dashboard-shell">` is the outer page container. Think of it as the full screen.
`<header className="hero">` is a semantic block for the title area. It is not only visual; it also improves structure.
`value={query}` means the input displays the current React state.
`onChange=(event) => setQuery(...)` means each keystroke updates state.
`events.map(...)` means the array is transformed into repeated visible cards.

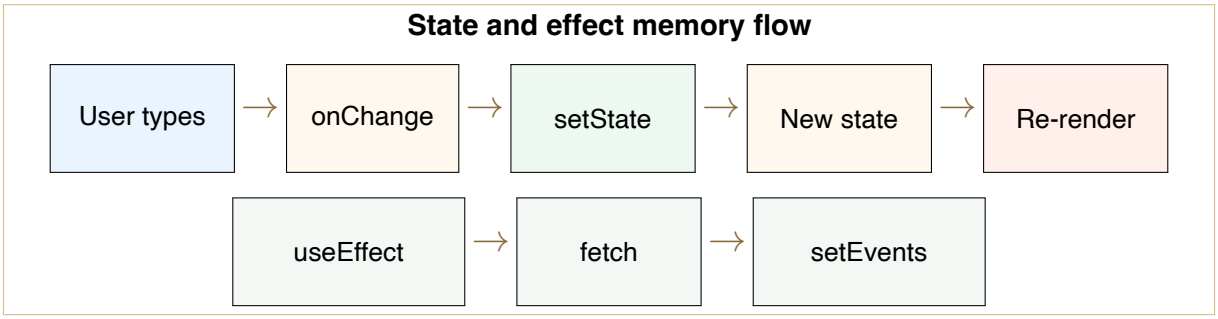
**Memory hook.** When JSX feels dense, do not start with the smallest tag. First ask: what is the page shell, what are the major sections, and what part repeats from data.

**Common mistake.** Many students memorize JSX as if it were magic HTML. That makes the code harder to remember. JSX is still JavaScript returning a UI tree. The tree is the important mental model.

## B.15 Step 9: understand `useState` as component memory

State is what makes React feel alive. The easiest way to remember it is this: `useState` gives the component memory between renders. A normal local variable disappears after a render. State survives

and tells React to render again when it changes.




Listing 20: One input and one boolean state

```
import { useState } from 'react'

function SearchPanel() {
  const [query, setQuery] = useState('')
  const [isOpen, setIsOpen] = useState(false)

  return (
    <section className="search-panel">
      <button onClick={() => setIsOpen(!isOpen)}>
        Toggle filters
      </button>

      {isOpen && (
        <input
          value={query}
          onChange={(event) => setQuery(event.target.value)}
          placeholder="Search events"
        />
      )}
    </section>
  )
}
```

| Moment | query | isOpen | Visible result |
|---|---|---|---|
| First render | empty string | false | button only |
| User clicks button | empty string | true | input appears |
| User types “par” | “par” | true | input shows “par” |

**Why this is memorable.**
`const [query, setQuery] = useState('')` creates one pair: the current state value and the function that updates it.
The component does not mutate the screen directly. It changes state.
React sees the new state and produces the new UI.
This is the core React sentence: **change state -> React re-renders -> UI updates**.

**Checklist.** When you see `useState`, ask four questions in order. What value is stored. What event changes it. Which JSX reads it. What visual change appears after the update.

**Common mistake.** Do not write `query = 'par'` inside a component and expect React to update the screen. Direct assignment does not trigger React rendering. Use the setter function.

## B.16 Step 10: understand `useEffect` and dependency arrays

`useEffect` is for work that happens because the component exists or because some external value changed. Fetching data from the backend is the most common example in this project. The important memory rule is: rendering describes the screen, while `useEffect` handles side effects around the screen.

Listing 21: A safe loading effect with explicit states

```
import { useEffect, useState } from 'react'

function EventsPage() {
  const [events, setEvents] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState('')

  useEffect(() => {
    let cancelled = false

    async function loadEvents() {
      try {
        setLoading(true)
        setError('')

        const response = await fetch('http://localhost:8000/api/events/')

        if (!response.ok) {
          throw new Error('Request failed')
        }

        const data = await response.json()

        if (!cancelled) {
          setEvents(data)
        }
      } catch (err) {
        if (!cancelled) {
          setError('Unable to load events')
        }
      } finally {
        if (!cancelled) {
          setLoading(false)
        }
      }
    }

    loadEvents()

    return () => {
      cancelled = true
    }
  }, [])
}
```

| Dependency form | Meaning |
|---|---|
| `[]` | run once after the first render |
| `[query]` | run again when `query` changes |
| no dependency array | run after every render |

Listing 22: Visible states the user can see

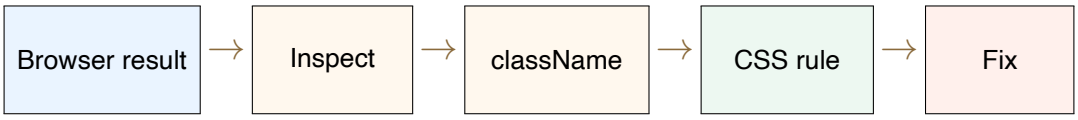
```
Loading events...
[ list of cards after success ]
[ Unable to load events ] after failure
```

**How to memorize this effect.**
The first render happens with empty data.
Then the effect runs and asks the backend for JSON.
When the JSON arrives, the component updates state.
That state update triggers a new render with real data on screen.
So the sequence is: **first render -> effect -> fetch -> state update -> second render**.

**Common mistake.** The classic `useEffect` bug is an infinite loop. It appears when the effect changes a state value and that same value is also a dependency in a way that retriggers the effect forever.

## B.17 Step 11: debug CSS classes, spacing, and flat-looking divs

When a frontend looks wrong, the best debugging order is visual, not emotional. Start from what the browser shows, inspect the element, verify the class name, check which rule is applied, and only then change CSS. This method is easier to remember than random guessing.




Read the page backward: what is visible in the browser, which class is applied, and which stylesheet rule must change.

Listing 23: HTML or JSX side

```
<article className="event-card featured-card">
  <h3>Spring Hackathon</h3>
  <p>Paris - April 21</p>
</article>
```

Listing 24: Broken CSS selector that explains why nothing seems to happen

```
.eventCard {
  padding: 1rem;
  border-radius: 16px;
  background: #fff8ee;
}
```

Listing 25: Correct selector that matches the actual class name

```
.event-card {
  padding: 1rem;
  border: 1px solid #d7b98a;
  border-radius: 16px;
  background: #fff8ee;
}
```

Listing 26: What the learner should notice

```
Broken: plain text on the page, no cream card, no border
Fixed : cream card, rounded corners, visible spacing
```

**Why beginners lose time here.**
If the class name in JSX is `event-card` but the stylesheet uses `.eventCard`, the rule will not match.
If the selector is correct but the page still looks flat, then inspect padding, margin, background, and border one by one.
Reading from the browser result back to the CSS rule is the fastest debugging pattern.

**Checklist.** Use this four-step repair method for any flat or broken layout.
1. Is the right class attached in JSX or HTML?
2. Does the selector in CSS match exactly?
3. Does the element have visible padding, margin, border, or background?
4. Does the responsive rule change the layout at the expected breakpoint?

## B.18 Step 12: derive filtered UI instead of storing duplicate state

One of the most important React habits is to store the smallest necessary state and derive the rest. In a search page, the real state is usually the original list plus the query. The filtered list is computed from those two values. It does not need its own permanent state variable.

Listing 27: A compact but realistic filtered page

```
import { useState } from 'react'

function EventsScreen({ events }) {
  const [query, setQuery] = useState('')

  const filteredEvents = events.filter((event) => {
    const haystack = `${event.title} ${event.location}`.toLowerCase()
    return haystack.includes(query.toLowerCase())
  })

  return (
    <main className="events-screen">
      <input
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        placeholder="Search title or location"
      />
```

```
        <p className="helper-text">
          {filteredEvents.length} result(s)
        </p>

        <section className="event-grid">
          {filteredEvents.map((event) => (
            <EventCard
              key={event.id}
              title={event.title}
              location={event.location}
            />
          ))}
        </section>
      </main>
    )
  }
```

Listing 28: Visible behavior

```
Query: "par"
Visible cards: events whose title or location contains "par"
Result count updates immediately after each keystroke
```

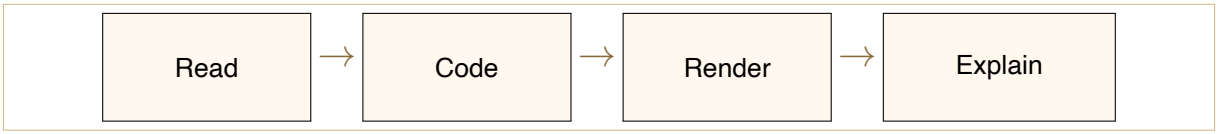
**Code reading focus.**
`query` is state because it changes over time.
`filteredEvents` is not state here; it is derived from `events` and `query`.
That makes the component easier to reason about because there is only one source of truth for the search text.

**Memory hook.** When React state starts to feel too big, ask: is this value truly stored, or can it be computed from existing state. That one question prevents many bugs.

## B.19 Worked Active Learning: Frontend

This active-learning block contains 15 frontend exercises. Each one asks a real development question, gives a short code target, shows the expected visual result, and explains the engineering idea behind the step.

**How to use this section.** Every exercise below is already solved. Read it in this order: question, exact code answer, expected browser result, then explanation. If you are a beginner, copy the solved code into a small practice file and run it before moving to the next exercise.




### Exercise 1: Semantic page shell

**Question.** How do you create a readable page before writing any CSS?

```
<main>
  <header>
    <h1>EventHub</h1>
    <p>Manage events and registrations.</p>
  </header>

  <section>
    <h2>Upcoming events</h2>
  </section>
</main>
```

```
+----------------------------------+
| EventHub                         |
| Manage events and registrations. |
| Upcoming events                  |
+----------------------------------+
```

**Explanation.** This is the HTML-first mindset. If the structure is already clear with plain text, the later React and CSS layers stay easier to understand and debug.

### Exercise 2: Event card styling

**Question.** How do you visually separate one event from another?

```
.event-card {
  padding: 1rem;
  border: 1px solid #d7b98a;
  border-radius: 16px;
  background: #fff8ee;
}
```

```
+------------------------------+
| Spring Hackathon             |
| Paris, April 21              |
+------------------------------+
```

**Explanation.** A card is a basic UI pattern. The border, padding, and background create separation so the user can scan content without confusion.

### Exercise 3: Responsive two-column grid

**Question.** How do you show two columns on desktop and one on mobile?

```
.grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 1rem;
}

@media (max-width: 700px) {
  .grid {
    grid-template-columns: 1fr;
```

```
  }
}
```

```
Desktop: [Card 1] [Card 2]
Mobile:
[Card 1]
[Card 2]
```

**Explanation.** Responsiveness is one of the assignment constraints. The media query changes layout without changing the content structure.

### Exercise 4: Primary button states

**Question.** How do you make an action button feel interactive?

```
.primary-btn {
  background: #182620;
  color: white;
  padding: 0.75rem 1rem;
  border-radius: 999px;
}

.primary-btn:hover {
  background: #2a3d34;
}
```

```
[ Create Event ]
hover -> darker background
```

**Explanation.** A hover state gives feedback before the click. That feedback is small, but it strongly improves usability.

### Exercise 5: Basic JavaScript data loop

**Question.** How do you inspect a list of events before rendering it?

```
const events = [
  { id: 1, title: 'Hackathon' },
  { id: 2, title: 'Workshop' },
]

for (const event of events) {
  console.log(event.title)
}
```

```
Hackathon
Workshop
```

**Explanation.** Before React renders anything, JavaScript still does the real data work. This exercise reinforces arrays, objects, and loops.

### Exercise 6: First reusable React component

**Question.** How do you avoid duplicating card markup?

```
function EventCard({ title, location }) {
  return (
    <article className="event-card">
      <h3>{title}</h3>
      <p>{location}</p>
    </article>
  )
}
```

```
+------------------+
| Hackathon        |
| Paris            |
+------------------+
```

**Explanation.** React components let one idea be written once and reused many times. That is the core reason React fits a SPA.

### Exercise 7: Render a list with `map`

**Question.** How do you turn data into repeated components?

```
const events = [
  { id: 1, title: 'Hackathon', location: 'Paris' },
  { id: 2, title: 'Meetup', location: 'Lyon' },
]

function EventList() {
  return events.map((event) => (
    <EventCard
      key={event.id}
      title={event.title}
      location={event.location}
    />
  ))
}
```

```
+------------------+  +------------------+
| Hackathon        |  | Meetup           |
| Paris            |  | Lyon             |
+------------------+  +------------------+
```

**Explanation.** Most CRUD screens are list screens. This exercise shows the standard React pattern for transforming arrays into visible UI.

### Exercise 8: Controlled input with `useState`

**Question.** How do you keep an input synchronized with React state?

```
import { useState } from 'react'
```

```
function SearchBox() {
  const [query, setQuery] = useState('')

  return (
    <input
      value={query}
      onChange={(event) => setQuery(event.target.value)}
      placeholder="Search events"
    />
  )
}
```

```
Type: "par"
State value: "par"
```

**Explanation.** A controlled input means React owns the value. That makes filtering, validation, and form submission predictable.

### Exercise 9: Filter visible events

**Question.** How do you show only events that match the search text?

```
const filteredEvents = events.filter((event) =>
  event.title.toLowerCase().includes(query.toLowerCase())
)
```

```
Query: "work"
Visible result:
Workshop
```

**Explanation.** Filtering is a small but real feature. It also teaches derived data: the UI is computed from the original state, not copied into new permanent state.

### Exercise 10: Load data from the API

**Question.** How does the frontend fetch real backend data?

```
import { useEffect, useState } from 'react'

function EventsPage() {
  const [events, setEvents] = useState([])

  useEffect(() => {
    fetch('http://localhost:8000/api/events/')
      .then((response) => response.json())
      .then(setEvents)
  }, [])
}
```

```
GET /api/events/
200 OK
```

```
[{"id":1,"title":"Spring Hackathon"}]
```

**Explanation.** This is the moment when the SPA stops being static. The browser becomes a client of the Django API.

### Exercise 11: Add a loading state

**Question.** What should users see while waiting for the server?

```
const [loading, setLoading] = useState(true)

if (loading) {
  return <p>Loading events...</p>
}
```

```
Loading events...
```

**Explanation.** A loading state explains delay instead of showing a blank page. It is a required quality pattern in serious SPAs.

### Exercise 12: Add an error state

**Question.** What should the interface do if the API fails?

```
const [error, setError] = useState('')

fetch('http://localhost:8000/api/events/')
  .catch(() => setError('Unable to load events'))

if (error) {
  return <p className="error-box">{error}</p>
}
```

```
[ Unable to load events ]
```

**Explanation.** Robust UIs must handle failure. This exercise teaches that API integration is not only about the success path.

### Exercise 13: Build a controlled form

**Question.** How do you collect new participant data in React?

```
const [form, setForm] = useState({ firstName: '', email: '' })

function updateField(event) {
  setForm({
    ...form,
    [event.target.name]: event.target.value,
  })
}
```

```
firstName = "Taylor"
email = "taylor@example.com"
```

**Explanation.** Real forms usually have multiple fields. This pattern scales better than writing one state variable per input.

### Exercise 14: Submit a POST request

**Question.** How do you send a new participant to the backend?

```
await fetch('http://localhost:8000/api/participants/', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(form),
})
```

```
POST /api/participants/
201 Created
```

**Explanation.** This is the CRUD creation path. It connects frontend form state to a real backend action.

### Exercise 15: Assemble a mini dashboard

**Question.** How do the small pieces become one SPA screen?

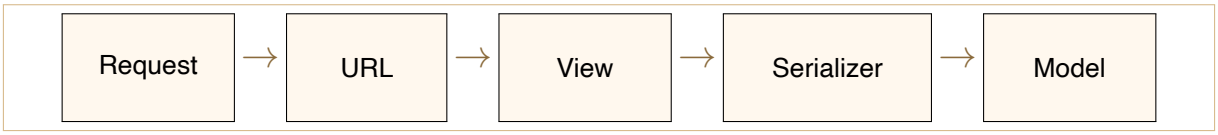
```
function Dashboard() {
  return (
    <main>
      <SearchBox />
      <section className="grid">
        <EventCard title="Hackathon" location="Paris" />
        <EventCard title="Meetup" location="Lyon" />
      </section>
    </main>
  )
}
```

```
+-------------------------------------+
| Search events [.................]    |
| [Hackathon | Paris] [Meetup | Lyon] |
+-------------------------------------+
```

**Explanation.** A SPA is built by composing small reliable pieces. This final frontend exercise shows how layout, state, and components come together into one page.

## C Backend Learning Path

## C.1 Basic concepts




| Term | Meaning |
|---|---|
| API | A set of endpoints that frontend and backend use to communicate |
| Django | Python framework that provides models, admin, views, and strong structure |
| Node.js | JavaScript runtime for backend execution |
| Express | Lightweight backend framework built on Node.js |
| Serializer | Converts model objects to JSON and validates incoming request data |
| Model | Python class that represents a database table |

**Memory hook.** Backend reasoning is easiest to remember as one chain: **request -> route -> validation -> database -> JSON response**. If one part breaks, inspect the chain from left to right.

## C.2 What the backend technologies do

**Django** is responsible for the main backend of EventHub. It stores the data model, applies the business rules, and exposes the official REST API.

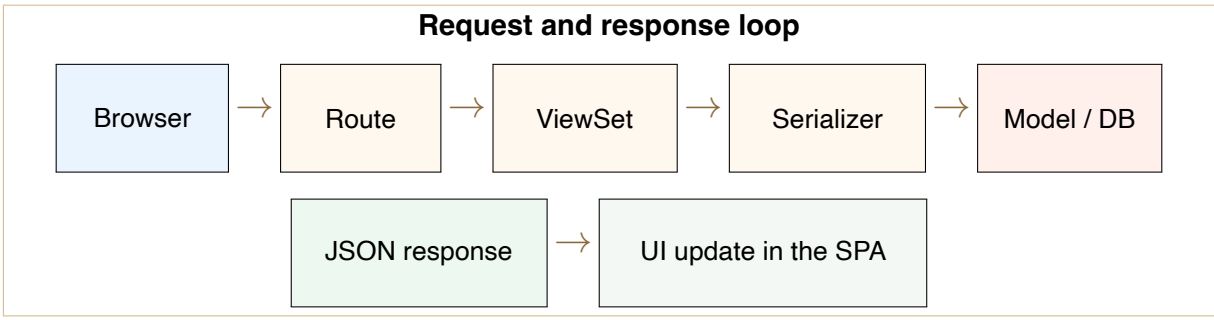
**Node.js** lets JavaScript run outside the browser.

**Express** makes it easier to build routes and middleware on top of Node.js.

**The API** is the contract between frontend and backend. For example, when the frontend calls `GET /api/events/`, the backend must return structured JSON.

## C.3 How one request travels through the backend

The backend becomes much easier to remember when it is visualized as a journey. A request starts in the browser, reaches a route, enters a view, passes through validation, reads or writes data, and then returns JSON to the frontend.




Listing 29: One complete request-response story

```
Browser -> GET /api/events/
Route matches /api/events/
ViewSet reads Event objects
```

```
Serializer converts Python objects to JSON
Server -> 200 OK
[
  { "id": 1, "title": "Spring Hackathon" }
]
React receives JSON and updates the page
```

**Memory sentence.** The browser does not talk directly to the database. It talks to the API, and the API decides what data becomes JSON.

## C.4 Mini Active Learning: backend request flow

**Question 1.** Why is a serializer important even if the model already exists?
**Answer.** Because the serializer controls how Python objects become JSON and how incoming request data is validated before saving.

**Question 2.** What comes back to the frontend after a successful backend request?
**Answer.** A structured HTTP response, often with JSON and a success status such as `200 OK` or `201 Created`.

**Question 3.** Why should the browser never talk directly to the database?
**Answer.** Because the backend must protect security, validation rules, permissions, and response formatting.

## C.5 Step 1: model the real entities

The project requires three main entities: events, participants, and registrations.

Listing 30: Core Django models

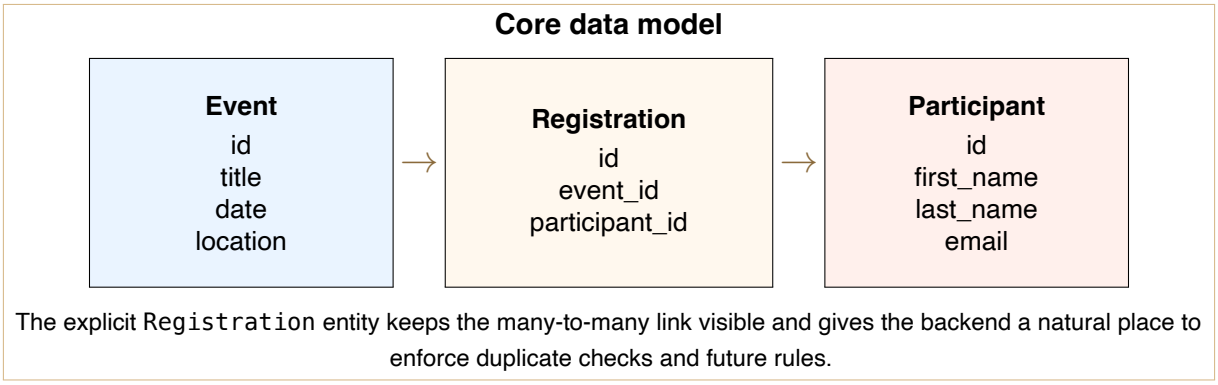
```python
class Event(models.Model):
    title = models.CharField(max_length=200)
    date = models.DateTimeField()
    location = models.CharField(max_length=200)


class Participant(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)


class Registration(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE)
    participant = models.ForeignKey(Participant, on_delete=models.CASCADE)
```

This structure is critical because the assignment explicitly requires a dedicated many-to-many relationship through a registration model.

**Line-by-line explanation.**
`class Event(models.Model)` means: create a database-backed Django model named Event.
`CharField` stores short text such as a title or a location.
`DateTimeField` stores both the date and the time.
`ForeignKey(..., on_delete=models.CASCADE)` means a registration points to one event or participant, and if the parent is deleted, its registrations are deleted too.

## C.6 How Event, Registration, and Participant fit together

Students often understand the three model names separately but forget how they connect. The easiest memory trick is to think of `Registration` as the bridge table in the middle.




**Memory sentence.** An event does not directly store all participant details, and a participant does not directly store all event details. The `Registration` model links them cleanly and can store business rules around that link.

## C.7 Step 2: run migrations

After creating or changing models, Django migrations update the database schema.

Listing 31: Migration commands

```
python manage.py makemigrations
python manage.py migrate
```

Listing 32: Expected result

```
Migrations for 'api':
  api/migrations/0001_initial.py
Applying api.0001_initial... OK
```

## C.8 Step 3: create serializers

Serializers make the API usable because they transform Python objects into JSON and validate requests.

Listing 33: Example serializer

```
class EventSerializer(serializers.ModelSerializer):
    class Meta:
        model = Event
        fields = "__all__"
```

## C.9 Step 4: expose the API through views and routes

Listing 34: Simple Django REST ViewSet

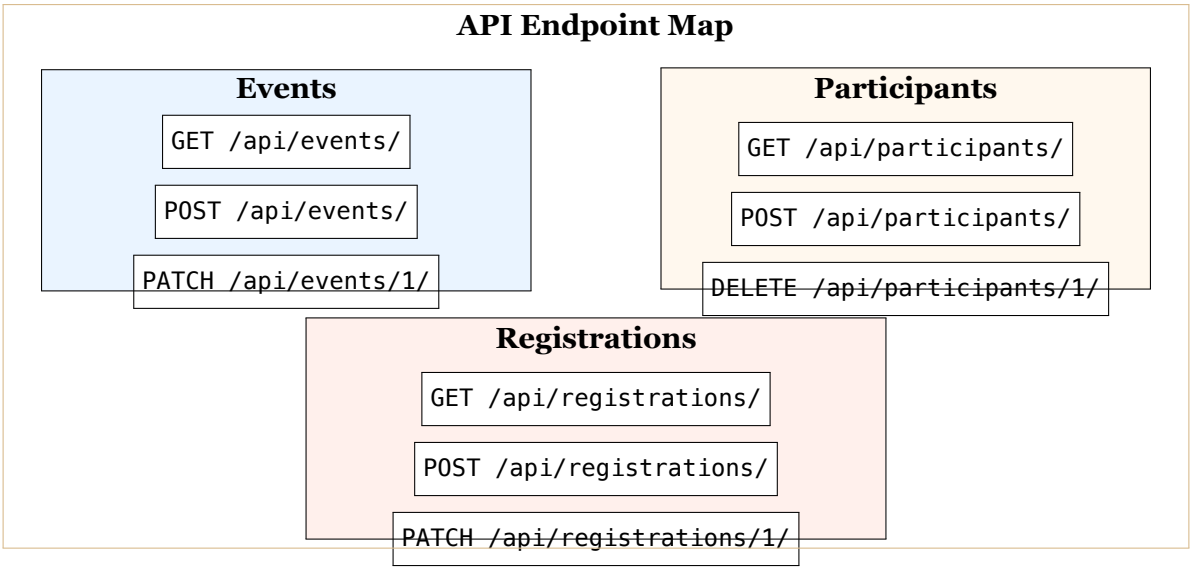
```
class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all().order_by("id")
    serializer_class = EventSerializer
```

Listing 35: Router registration

```
router = DefaultRouter()
router.register(r"events", EventViewSet)
router.register(r"participants", ParticipantViewSet)
router.register(r"registrations", RegistrationViewSet)
```

Listing 36: Expected endpoint examples

```
GET    /api/events/
POST   /api/events/
GET    /api/participants/
POST   /api/registrations/
```




## C.10 Step 5: understand validation

The backend must protect the application rules. For example, one participant cannot register twice for the same event.

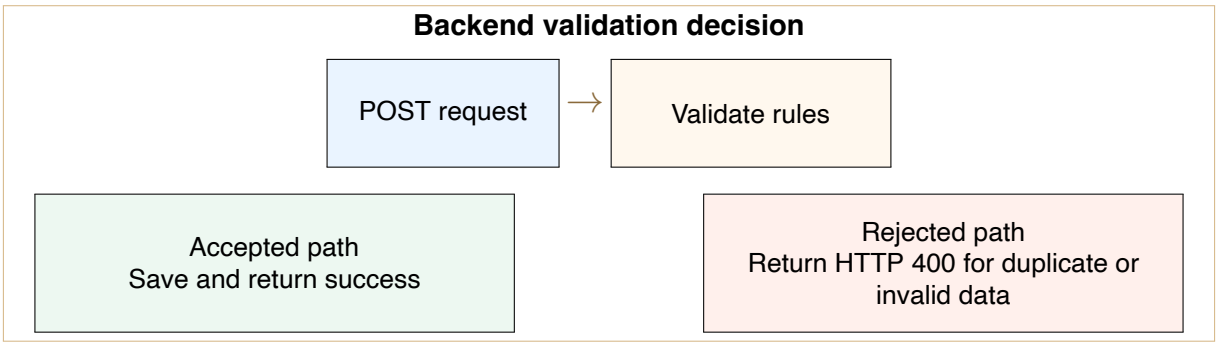
Listing 37: Business validation example

```
if duplicate_qs.exists():
    raise serializers.ValidationError(
        "This participant is already registered for this event."
```

```
    )
```

Listing 38: Expected error response idea

```
{
  "error": true,
  "status_code": 400,
  "details": {
    "non_field_errors": [
      "This participant is already registered for this event."
    ]
  }
}
```




## C.11 Mini Active Learning: backend validation

**Question 1.** Why is duplicate-registration protection a backend concern and not only a frontend concern?
**Answer.** Because frontend checks can be bypassed. The backend is the final authority and must enforce business rules for every request.

**Question 2.** What should the API return when validation fails?
**Answer.** A clear error response, typically with a `400` status and a message explaining what rule was violated.

**Question 3.** What is the mental model for validation?
**Answer.** Every incoming write request must pass through a decision gate: valid data is saved, invalid data is rejected.

## C.12 Step 6: build the Node.js and Express comparison backend

The comparative backend should stay smaller than Django. Its job is not to replace Django. Its job is to make the comparison concrete. In the final workspace, the Express service runs on port `3001`, stores its own data in `node/db.sqlite3`, and focuses on events and participants rather than authentication and registrations.

Listing 39: Local Node backend startup

```
cd node
export PATH="/opt/homebrew/opt/node@22/bin:$PATH"
npm install
npm run dev
```

Listing 40: Minimal Express server

```
import express from 'express'
import cors from 'cors'

const app = express()
const port = process.env.PORT || 3001

app.use(cors())
app.use(express.json())

app.get('/api/health', (req, res) => {
  res.json({
    status: 'ok',
    service: 'eventhub-node-backend',
  })
})

app.listen(port, () => {
  console.log(`Node backend listening on http://localhost:${port}`)
})
```

Listing 41: Expected JSON output

```
GET http://localhost:3001/api/health

{
  "status": "ok",
  "service": "eventhub-node-backend"
}
```

## C.13 Step 7: compare Django and Express clearly

Django and Express can both expose a CRUD API, but they do not guide the developer in the same way. Django starts with a stricter structure: projects, apps, models, serializers, URLs, and migrations all have a clear place. Express is more open. That flexibility can feel simpler at the beginning, but it also means the team has to decide more things manually.

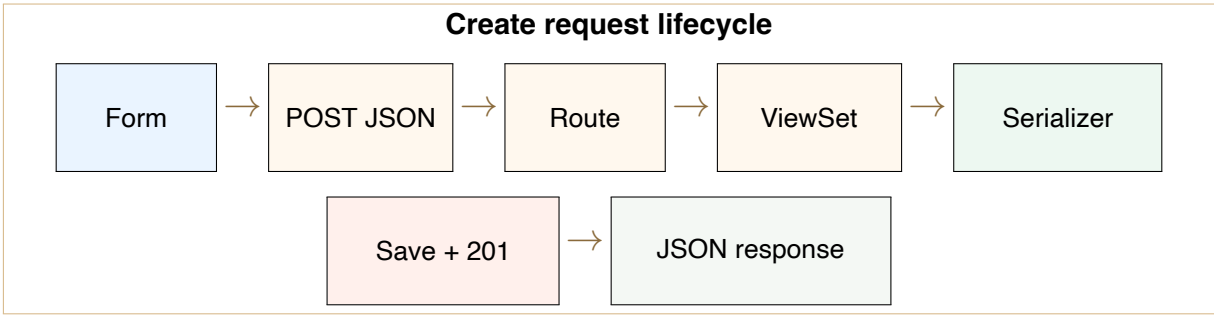
| Criterion | Django | Node.js / Express |
|---|---|---|
| Project structure | Project and app layout enforced by the framework | File layout is chosen manually by the developer |
| Framework style | Batteries included and opinionated | Minimal and unopinionated |
| Schema changes | Migrations track database changes clearly | SQL changes are manual unless extra tools are added |
| Validation | Strong serializer and model flow | Often written directly in route handlers or middleware |
| Admin and auth | Built-in admin and auth support | Must be added with extra packages or custom code |
| Scalability style | Good for model-heavy business applications | Good for I/O-heavy APIs and lightweight services |
| Project role here | Main backend | Comparative backend |
| Runtime note | Python virtual environment | Node.js 22 for native SQLite dependency |

**Memory comparison.** Django gives a strong default structure for business applications. Express gives a lighter starting point but requires more manual organization. For EventHub, that is why Django is the main backend and Express is the comparison backend.

Another useful way to remember the difference is this: Django gives more tools at the start, while Express gives more freedom at the start. In a student project, Express can feel quick because the first routes are easy to write in JavaScript. But as the project grows, Django often saves time because validation, authentication, admin support, and schema tracking already have a clear place in the framework. The Node.js event loop is still an advantage for highly concurrent I/O-heavy services, which is why Node remains an important comparison case in web development.

## C.14 Step 8: read one POST request from the form to the database

The easiest way to remember backend logic is to follow one concrete write request. Imagine the user fills the participant form and clicks submit. The browser sends JSON, the route sends the request to the correct view, the serializer validates the data, the model is saved, and the backend answers with a success response.




Listing 42: One realistic creation request

```
POST /api/participants/
Content-Type: application/json

{
  "first_name": "Taylor",
  "last_name": "Rivera",
  "email": "taylor@example.com"
}
```

Listing 43: The viewset receives the request and delegates the heavy work

```
class ParticipantViewSet(viewsets.ModelViewSet):
    queryset = Participant.objects.all().order_by('id')
    serializer_class = ParticipantSerializer
```

Listing 44: Successful response

```
201 Created
{
  "id": 3,
  "first_name": "Taylor",
  "last_name": "Rivera",
  "email": "taylor@example.com"
}
```

**What actually happened.**
The browser did not create the database row directly.
It sent JSON to the API endpoint.
The viewset selected the serializer and the model flow for that endpoint.
The serializer checked whether the fields were acceptable.
Django saved the new model row and returned JSON describing the created object.

**Memory hook.** For write requests, memorize one chain only: **form -> JSON -> route -> serializer -> model save -> response**. If you remember that chain, most backend code becomes easier to place mentally.

## C.15 Step 9: understand the serializer lifecycle

Students often know that a serializer exists, but they do not know what job it actually performs. The serializer is the conversion and validation gate. Incoming request data enters as raw JSON-like data. The serializer checks it, transforms it into validated Python data, and then creates or updates model instances.

Listing 45: A serializer with explicit business validation

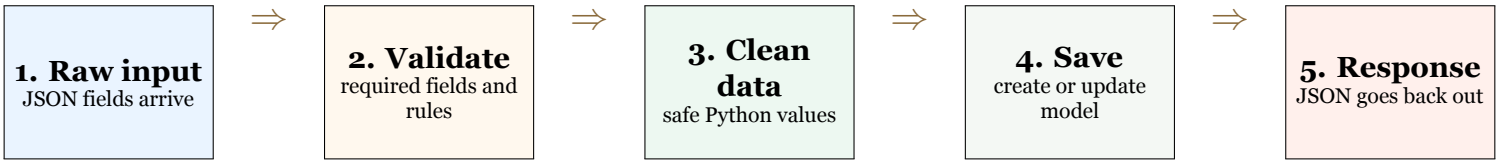
```
class RegistrationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Registration
        fields = '__all__'

    def validate(self, attrs):
        duplicate = Registration.objects.filter(
            event=attrs['event'],
            participant=attrs['participant'],
        ).exists()

        if duplicate:
            raise serializers.ValidationError(
                'This participant is already registered for this event.'
            )

        return attrs
```

| Serializer stage | What it means in plain language |
|---|---|
| Raw input | request data arrives from the browser as JSON fields |
| Validation | the serializer checks required fields and business rules |
| Validated data | the backend now trusts the cleaned values |
| Save | Django creates or updates the model row |
| Response | the serializer turns the saved object back into JSON |

Listing 46: Validation failure that the frontend must understand

```
400 Bad Request
{
  "non_field_errors": [
```

```
      "This participant is already registered for this event."
    ]
  }
```

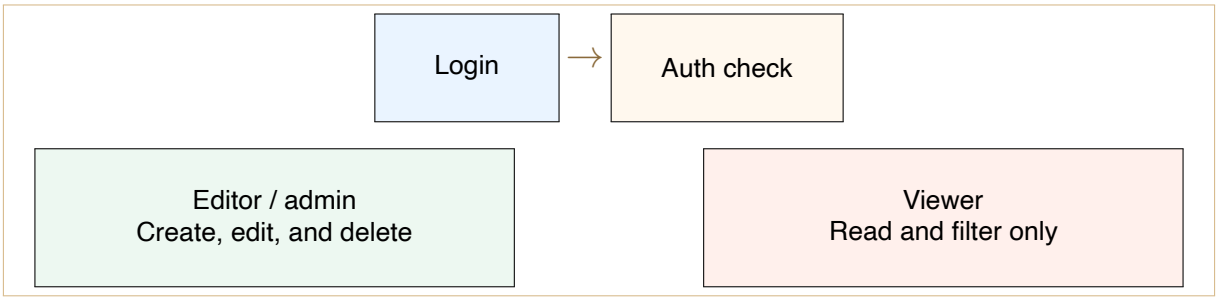



**Common mistake.** A frequent beginner error is to think that the serializer is only for output formatting. In reality it also protects the backend by validating incoming data before save.

**Checklist.** When reading serializer code, ask in this order. Which model is connected. Which fields are exposed. Which custom rules are checked. What JSON shape will return to the frontend.

## C.16 Step 10: understand login, permissions, and role-based behavior

Role-based behavior is one of the most important defense topics because the jury may ask you to prove that not all users can do the same actions. Even before a full production authentication system is finished, the concept should be understood completely.




Listing 47: Read-only access for anonymous users, write access for authenticated users

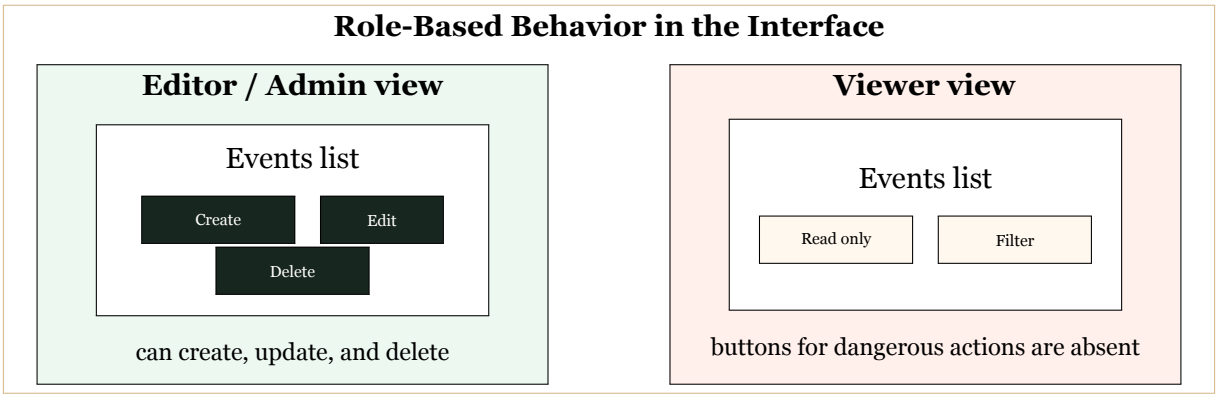
```
from rest_framework.permissions import IsAuthenticatedOrReadOnly

class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all().order_by('id')
    serializer_class = EventSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```

Listing 48: One simple custom role permission example

```
from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsEditorOrReadOnly(BasePermission):
    def has_permission(self, request, view):
        if request.method in SAFE_METHODS:
            return True
```

```
        return (
            request.user.is_authenticated
            and request.user.groups.filter(name='editor').exists()
        )
```

| User type | Allowed actions | Visible frontend effect |
|---|---|---|
| Anonymous or viewer | login, list, search, read details | create buttons hidden or disabled |
| Editor or admin | read plus create, update, delete | forms and action buttons available |




**What to say during the defense.**
Authentication answers: who is this user.
Permissions answer: what is this user allowed to do.
Role-based behavior means the UI and the API must agree. If the frontend hides a button but the backend still allows everyone to post, the protection is incomplete.

## C.17 Step 11: compare the same endpoint in Django and Express

The cleanest comparison between Django and Express is to implement the same tiny endpoint in both technologies and read them side by side. The important lesson is not only that the syntax differs. The deeper lesson is that Django gives more structure by default, while Express gives more manual control.

Listing 49: One health endpoint in Django REST Framework

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(['GET'])
def health(request):
    return Response({
        'status': 'ok',
        'service': 'eventhub-django-backend',
    })
```

Listing 50: The same idea in Express

```
app.get('/api/health', (req, res) => {
  res.json({
    status: 'ok',
```

```
    service: 'eventhub-node-backend',
  })
})
```

| Reading point | Django | Express |
|---|---|---|
| Runtime language | Python | JavaScript |
| Routing style | decorator or router and viewset system | direct route callback |
| Validation ecosystem | built into a broader framework flow | chosen manually per project |
| Typical feeling | more structure up front | more freedom up front |

**Framework comparison at a glance**

**Django**

- Python framework with strong conventions
- Models, serializers, migrations, and admin
- Fast CRUD delivery for structured business data strongest for applications with explicit entities and validation rules

**Node.js / Express**

- JavaScript runtime plus lightweight routing
- Manual middleware, CORS, and route structure
- Events and participants API on port 3001 strongest when low ceremony and custom composition matter more than built-ins

**Memory hook.** Node.js is the runtime. Express is the lightweight framework. Django is a full Python framework. Remembering these three roles clearly prevents many oral-defense mistakes.

## C.18 Step 12: debug the backend without panic

When a backend request fails, the right method is to debug in layers. Do not jump immediately into React. First verify that the server runs, then that the route exists, then that the request body is correct, then that validation passes, and only after that inspect the frontend integration.

Listing 51: Fast backend verification commands

```
python manage.py runserver
python manage.py makemigrations
python manage.py migrate
curl http://localhost:8000/api/events/
cd node
export PATH="/opt/homebrew/opt/node@22/bin:$PATH"
npm run dev
curl http://localhost:3001/api/health
```

| Symptom | Likely cause | First check |
|---|---|---|
| 404 on an endpoint | wrong URL or missing router entry | API route registration |
| Connection refused | backend process is not running or the wrong port is used | confirm `:8000` for Django and `:3001` for Express |
| 400 on POST | validation or missing field | serializer errors and request body |
| 401 or 403 | authentication or permission failure | login state and permission classes |
| 500 server error | exception in code | terminal traceback and recent code changes |
| Frontend shows nothing | request failed before render | test the endpoint with `curl` first |

**Checklist.** Use this order every time. Is the right server running on the right port. Does the endpoint exist. Does the request body match the serializer or route fields. Does validation accept the data. Does the JSON response match what React expects. If the endpoint is `/api/auth/...` or `/api/registrations/...`, check Django. If the endpoint is the Express comparison API, check Node on `:3001`.

## C.19 Mini Active Learning: backend comparison

**Question 1.** Why is Django the better main backend choice for this project?
**Answer.** Because the project needs relational models, validation, admin support, and a structured CRUD API, all of which Django provides efficiently.

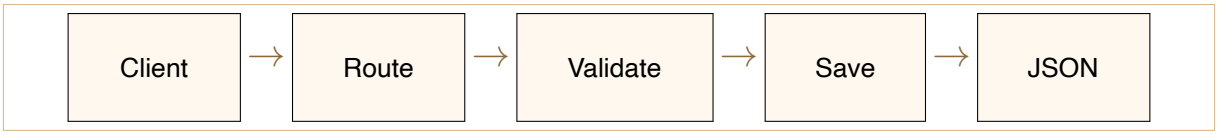
**Question 2.** Why keep Node.js and Express at all?
**Answer.** Because the course asks for a comparison and Express is useful for understanding a more manual, lightweight backend style.

**Question 3.** What should be memorized for the oral defense?
**Answer.** Django = structured full framework, Express = minimal flexible framework, Node.js = runtime, API = communication contract.

## C.20 Worked Active Learning: Backend

This active-learning block contains 15 backend exercises. It moves from core Django modeling and API flow to a small Express comparison so the report teaches both implementation and reasoning.

**How to use this section.** Every backend exercise below is fully solved. Do not read it as a homework sheet. Read it as a guided correction: identify the goal, study the exact code answer, then compare the JSON or terminal result with the explanation.




### Exercise 16: Identify the first API endpoint

**Question.** What is the simplest useful backend endpoint for EventHub?

```
GET /api/events/
```

```
[
  {
    "id": 1,
    "title": "Spring Hackathon"
  }
]
```

**Explanation.** Starting with a read-only endpoint reduces complexity. It lets the frontend team connect early, even before create and update actions exist.

### Exercise 17: Define the `Event` model

**Question.** Which fields does one event need at minimum?

```
class Event(models.Model):
    title = models.CharField(max_length=200)
    date = models.DateTimeField()
    location = models.CharField(max_length=200)
```

```
Table created: api_event
Columns: id, title, date, location
```

**Explanation.** Backend design starts with domain entities. If the model is weak, every serializer, view, and frontend screen becomes harder to build.

### Exercise 18: Define the `Participant` model

**Question.** How do you represent a person who can join many events?

```
class Participant(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
```

```
Table created: api_participant
Unique field: email
```

**Explanation.** The unique email constraint is a real business rule. It prevents multiple participant records that actually describe the same person.

### Exercise 19: Create the `Registration` relation

**Question.** Why use a dedicated registration table instead of a direct many-to-many field only?

```
class Registration(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE)
    participant = models.ForeignKey(Participant, on_delete=models.CASCADE)
```

```
Event 1 <-> Participant 5
stored as one explicit registration row
```

**Explanation.** A dedicated relation is more academic and more practical. It allows later fields such as status, registration date, or payment state.

### Exercise 20: Prevent duplicate registrations

**Question.** How do you stop the same participant from joining the same event twice?

```
class Registration(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE)
    participant = models.ForeignKey(Participant, on_delete=models.CASCADE)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=['event', 'participant'],
                name='unique_event_participant',
            )
        ]
```

```
Duplicate insert -> IntegrityError or validation failure
```

**Explanation.** Constraints protect data even when the frontend makes mistakes. That is one of the main jobs of a serious backend.

### Exercise 21: Run migrations

**Question.** What turns model code into real database tables?

```
python manage.py makemigrations
python manage.py migrate
```

```
Migrations for 'api':
  api/migrations/0001_initial.py
Applying api.0001_initial... OK
```

**Explanation.** Migrations are the bridge between Python definitions and the real database schema. Without them, the backend models exist only in source code.

### Exercise 22: Serialize model data

**Question.** How does Django convert model objects into JSON?

```
class EventSerializer(serializers.ModelSerializer):
    class Meta:
        model = Event
```

```
        fields = '__all__'
```

```
{
  "id": 1,
  "title": "Spring Hackathon",
  "location": "Paris"
}
```

**Explanation.** Serializers are the translation layer of the API. They also become the natural place for request validation.

### Exercise 23: Add business validation

**Question.** Where should custom registration checks live?

```
def validate(self, attrs):
    if Registration.objects.filter(
        event=attrs['event'],
        participant=attrs['participant'],
    ).exists():
        raise serializers.ValidationError(
            'Participant already registered.'
        )
    return attrs
```

```
400 Bad Request
{"non_field_errors":["Participant already registered."]}
```

**Explanation.** This exercise teaches the difference between data shape and business rule. A request can be syntactically valid and still violate domain logic.

### Exercise 24: Expose CRUD with a viewset

**Question.** How do you create one class that supports list, create, retrieve, update, and delete?

```
class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all().order_by('id')
    serializer_class = EventSerializer
```

```
GET    /api/events/
POST   /api/events/
PATCH  /api/events/1/
DELETE /api/events/1/
```

**Explanation.** Django REST Framework reduces boilerplate. For course work, this is a strong reason to prefer Django as the primary backend.

### Exercise 25: Register API routes

**Question.** How do URLs connect to the viewsets?

```
router = DefaultRouter()
router.register(r'events', EventViewSet)
router.register(r'participants', ParticipantViewSet)
router.register(r'registrations', RegistrationViewSet)
```

```
/api/events/
/api/participants/
/api/registrations/
```

**Explanation.** Routing is the public map of the backend. Clear route design helps both teammates and future testers.

### Exercise 26: Test the endpoint manually

**Question.** How can you verify the API before connecting React?

```
curl http://localhost:8000/api/events/
```

```
[{"id":1,"title":"Spring Hackathon","location":"Paris"}]
```

**Explanation.** Direct endpoint testing isolates backend bugs. If `curl` fails, the problem is not in React.

### Exercise 27: Protect the API

**Question.** How do you restrict modification endpoints to authenticated users?

```
from rest_framework.permissions import IsAuthenticatedOrReadOnly

class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```

```
Anonymous GET  -> 200 OK
Anonymous POST -> 401 Unauthorized
```

**Explanation.** Permissions are part of the assignment logic, not an optional detail. A project with roles needs route protection from the start.

### Exercise 28: Start a minimal Express server

**Question.** What is the smallest valid Node.js and Express backend for comparison, and how should it be started in this project?

```
cd node
export PATH="/opt/homebrew/opt/node@22/bin:$PATH"
npm run dev
```

```
import express from 'express'

const app = express()
const port = process.env.PORT || 3001

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', service: 'eventhub-node-backend' })
})

app.listen(port)
```

```
GET http://localhost:3001/api/health
{"status":"ok","service":"eventhub-node-backend"}
```

**Explanation.** This demonstrates the Express philosophy: less built-in structure, faster start, and more manual control. In this repository the Node backend should be run with Node.js 22 because `better-sqlite3` is a native dependency.

### Exercise 29: Add a simple Express collection route

**Question.** How does the comparison backend return the events collection in Express?

```
import { Router } from 'express'
import db from '../db.js'

const router = Router()

router.get('/', (req, res) => {
  const events = db.prepare('SELECT * FROM events ORDER BY id').all()
  res.json(events)
})
```

```
GET http://localhost:3001/api/events/
[]
```

**Explanation.** This exercise shows how transparent Express is. The route callback directly reads from SQLite and returns JSON. What you gain in simplicity, you later repay with more manual architecture, because authentication, registration filtering, admin tools, and advanced validation are not automatically provided.

### Exercise 30: Write the backend comparison

**Question.** After implementing both backends, what should the report conclude?

```
Django  -> stronger structure, admin, serializers, faster CRUD setup
Express -> lighter, flexible, easier to customize route by route
```

```
Decision for EventHub:
Main backend = Django
Comparison backend = Node.js / Express
```

**Explanation.** The final exercise is reflective on purpose. A project report should not only show code; it should justify technology choices with concrete evidence from implementation.

# D Project Construction Order

The best step-by-step order for this project is:

1. finalize the Django models and API,
2. prepare the smaller Node.js and Express comparison backend,
3. build the React pages and responsive layout,
4. connect the frontend to the Django API,
5. add authentication, roles, loading states, and route protection,
6. test the whole system,
7. deploy frontend and backend,
8. finish the report and defense material.

# E Memorization, Debugging, and Defense Atlas

This section is here to make revision easier before coding sessions, oral defense preparation, and the CC1 exam. The goal is not only to know the answer, but also to remember *why* it is correct and what to check when something breaks.

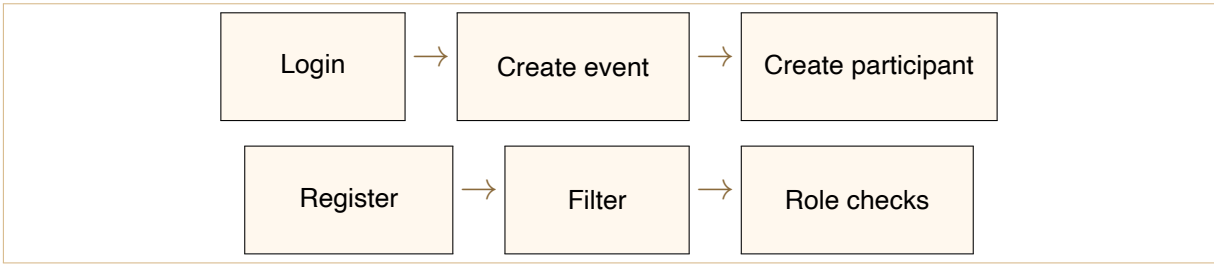
## E.1 One-sentence memory ladders

| Layer | One-sentence memory | Question to ask yourself |
|---|---|---|
| HTML | HTML gives the page its structure and meaning. | What content blocks exist on this screen? |
| CSS | CSS decides spacing, color, layout, and responsiveness. | Why does this box look wrong or collide with another box? |
| JavaScript | JavaScript reacts to user actions and transforms data. | Which event or condition changes the behavior? |
| React | React redraws the UI from state instead of manual DOM edits. | Which state changed, and which JSX reads it? |
| API | The API is the contract between frontend and backend. | What JSON shape is sent or returned here? |
| Django | Django gives strong structure for models, validation, and CRUD APIs. | Which model, serializer, and route are involved? |
| Node.js | Node.js is the runtime that executes JavaScript on the server. | Is this about the runtime or the framework built on top of it? |
| Express | Express is a lightweight routing and middleware framework. | What route and middleware chain handle this request? |

**Memory hook.** If you ever feel lost, reduce the problem to one layer. Ask first: is this structure, style, behavior, state, or API communication. That one question often tells you where to debug.

## E.2 Live demonstration storyboard

The defense requires a live demonstration, so it is worth preparing the exact story in advance. A good demo is short, calm, and structured. Each click should prove one requirement and one technical idea at the same time.




| Demo step | What you do live | What it proves technically |
|---|---|---|
| 1. Login | sign in with an editor or admin account | authentication works and the app can identify the user |
| 2. Create event | fill the event form and submit it | frontend form state, POST request, serializer, and database save all work |
| 3. Create participant | submit a participant form | another CRUD creation path works from UI to backend |
| 4. Register participant | connect one participant to one event | the relation model and business flow are correct |
| 5. Filter results | type inside the search input | JavaScript or React state changes update the visible list |
| 6. Role behavior | switch to a viewer account and compare the UI | permissions are enforced in both frontend behavior and backend policy |

**Minimal defense script.**
"First I log in as an editor. Now I create a new event. The event appears in the list, which shows the POST request and database save worked. Next I create a participant, then register that participant for the event. After that I filter the list with a search query. Finally I switch to a viewer account to show that creation actions are no longer available."
This script is simple on purpose. During a defense, clarity is stronger than showing too many features too quickly.

## E.3 What the examiner can ask after each click

| Action shown in the demo | Typical follow-up question and the right idea to answer |
|---|---|
| Login | How is the user recognized? Answer: by the authentication system, session or token, and backend permission checks. |
| Create event | What endpoint receives this form? Answer: the event creation API endpoint using `POST`. |
| Create participant | Where is validation performed? Answer: in the backend serializer and model rules, not only in the frontend. |
| Register participant | Why use a separate registration model? Answer: because the relation itself is a first-class business entity. |
| Filter results | Is filtering done in frontend or backend? Answer: it can be frontend-side for already-loaded lists, or backend-side for larger datasets. |
| Role behavior | Why hide buttons if the backend already protects routes? Answer: hiding buttons improves UX, but backend protection remains the final authority. |

## E.4 Frontend debugging atlas

| Symptom | Likely frontend cause | Fast repair method |
|---|---|---|
| Nothing appears on screen | wrong component export, wrong import, or render error | check browser console and the root component import chain |
| Styles do not apply | class mismatch or stylesheet not imported | inspect the element and compare class names exactly |
| Cards touch each other | missing margin, gap, or parent layout spacing | check the container layout first, then the card spacing |
| Input does not update | missing `value` or `onChange` pairing | confirm the component is a controlled input |
| List does not refresh after typing | derived filtering logic not connected to state | verify which JSX reads the search state |
| API data never appears | fetch error or wrong endpoint URL | check loading and error states, then test the endpoint directly |
| Page reloads on submit | missing `preventDefault()` | stop the default form action before custom logic |
| Route change reloads the page | plain `<a>` used instead of client-side routing | use router navigation primitives for SPA behavior |

Listing 52: A simple frontend repair order

```
1. Read the browser result.
2. Open the console for errors.
3. Inspect the element.
4. Verify className and CSS selector match.
5. Verify state changes after the event.
6. Test the API separately if data is missing.
```

**Common mistake.** Do not debug three layers at once. If the UI is empty, first decide whether the problem is render logic, CSS visibility, or API loading. One layer at a time is faster and more memorable.

## E.5 Backend debugging atlas

| Symptom | Likely backend cause | Fast repair method |
|---|---|---|
| `404 Not Found` | route missing or wrong path prefix | verify router registration and the full API URL |
| `400 Bad Request` | validation error or missing field | read serializer errors and compare with request JSON |
| `401 Unauthorized` | user not authenticated | verify login and auth headers or session |
| `403 Forbidden` | authenticated but not allowed | inspect permission classes and user role |
| `500 Internal Server Error` | Python exception inside view, serializer, or model | read the traceback carefully from top to bottom |
| Duplicate relation created | missing unique constraint or validation rule | add constraint plus serializer validation |
| Frontend expects field not returned by API | serializer field mismatch | compare JSON response with frontend property names |

Listing 53: A simple backend repair order

```
1. Confirm the server is running.
2. Call the endpoint directly with curl.
3. Read the status code.
4. Read the JSON error body.
5. Read the serializer or permission class.
6. Only then reconnect the frontend.
```

**Checklist.** For backend debugging, the status code is a memory shortcut.
`404` means path or route.
`400` means data or validation.
`401/403` means authentication or permission.
`500` means code exception.

## E.6 Annotated mini walkthrough: login to protected action

Even if the authentication code changes later, the main idea stays the same. The flow below is the part that matters for the defense.

Listing 54: Protected-action story

```
User logs in
```

```
-> backend verifies credentials
-> frontend stores login state
-> user opens create-event form
-> frontend sends authenticated POST request
-> backend permission class checks the role
-> allowed user gets 201 Created
-> viewer gets 403 Forbidden
```

**Why this story matters.**
The project does not only need forms and lists. It also needs controlled behavior depending on who the user is.
That is why role-based behavior must be explained both in the UI and in the API layer.

## E.7 Exact frontend example: `withSessionGuard`

One concrete helper used in the frontend is `frontend/src/HOC/withSessionGuard.jsx`. This file is a small **Higher-Order Component (HOC)**. Here, that means a function that receives a component and returns a protected version of it.

Listing 55: Project route-protection helper

```jsx
import { createElement } from 'react'

export default function withSessionGuard(Component) {
  return function GuardedComponent({ fallback = null, session, ...props }) {
    if (!session?.accessToken) {
      return fallback
    }

    return createElement(Component, props)
  }
}
```

1. `withSessionGuard(Component)` receives a component as input.
2. It returns a new wrapper component called `GuardedComponent`.
3. The wrapper reads `session?.accessToken` to decide whether the user is authenticated.
4. If no access token exists, it returns the `fallback` view, which can be `null` or a public screen.
5. If an access token exists, it renders the original component.
6. `createElement(Component, props)` is the React call that performs that rendering.
7. Because `session` is removed from `...props`, the wrapped component does not receive it automatically. The session is used only for the guard decision.

**Why this helper is useful.**
It keeps route protection simple and reusable. Instead of repeating "if not logged in, block this page" in many different components, the project defines that logic once and reuses it where needed. In EventHub, this helper is connected in `App.jsx` to protect the main workspace shell. The exact usage appears in the declaration `const ProtectedWorkspaceShell = withSessionGuard(...)` before the protected interface renders `AppShell`.

### E.8 Memorable code-reading rules

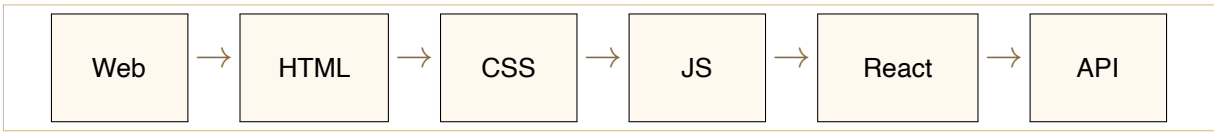
1. In HTML, read from the biggest semantic block to the smallest element.
2. In CSS, read selectors first, then layout properties, then spacing, then colors.
3. In JavaScript, find the event source first: click, submit, input, timer, or fetch.
4. In React, find the state variables before reading the JSX.
5. In backend code, find the endpoint first, then the serializer, then the model.
6. In debugging, trust visible evidence before guesses: status codes, console errors, and actual rendered output.

### E.9 Quick oral-defense answers to memorize

1. **Why React for the frontend?** Because the project requires a SPA with reusable components, state-driven UI updates, and easy API integration.
2. **Why Django for the main backend?** Because the project needs relational data modeling, validation, and fast CRUD setup with strong structure.
3. **Why keep Node.js and Express?** Because the assignment asks for comparison, and Express shows a lighter and more manual backend style.
4. **Why a separate Registration model?** Because the relationship between events and participants is itself an important business object.
5. **Why loading and error states?** Because asynchronous UI must explain what is happening to the user at all times.
6. **Why backend validation too?** Because frontend checks can be bypassed, so the backend must be the final authority.

## F Complete Independent Study Bank

This section reorganizes the learning path by topic so it can be used as a revision bank on its own. It goes from web foundations to React, APIs, and exam-style practice in a more direct order.



## F.1 Topic roadmap

| Topic area | What is included | How this section is used |
|---|---|---|
| Web architecture and deployment | client/server roles, HTTP, HTTPS, REST, stacks, deployment basics | use this first to understand how all other layers fit together |
| HTML structure and semantics | tags, links, images, forms, document structure, accessibility basics | use this before CSS and JavaScript practice |
| CSS and responsive design | selectors, box model, Flexbox, Grid, media queries, responsive thinking | use this with the frontend visual exercises and box-model examples |
| JavaScript and DOM | syntax drills, functions, loops, events, DOM selection, validation logic | use this before React hooks and API work |
| React fundamentals | components, props, state, lists, controlled inputs, SPA thinking | use this before advanced React and routing questions |
| React, APIs, and security | `useEffect`, loading/error states, JSON, JWT, routing, protected flows | use this before the final CC1 and defense preparation sections |
| Hands-on practice | frontend and backend exercises, practical tasks, and implementation order | use this after the theory bank to turn knowledge into execution |

Study strategy: work through these sections in order. Each topic block gives questions, answers, and explanations so the appendix can be used directly for revision from one PDF.

## F.2 Web architecture, REST, and deployment

1. **Question.** What is the main role of the client in web development?
   **Answer.** Display the interface and handle user interactions.
   **Explanation.** The client is the browser-facing layer. It renders the UI, captures clicks and form input, and sends requests to the server.

2. **Question.** Which component handles business logic and talks to the database?
   **Answer.** The server.
   **Explanation.** This is the backend role in the client-server model.

3. **Question.** What does HTTP stand for?
   **Answer.** HyperText Transfer Protocol.
   **Explanation.** It is the protocol used for request-response communication on the web.

4. **Question.** What is the key difference between HTTP and HTTPS?
   **Answer.** HTTPS encrypts the communication.
   **Explanation.** HTTPS adds TLS security, which protects transmitted data.

5. **Question.** What does REST stand for?
   **Answer.** Representational State Transfer.
   **Explanation.** REST is one of the dominant architectural styles for modern web APIs.

6. **Question.** Which option is *not* a REST principle?
   **Answer.** Keeping hidden client session state on the server for every request.
   **Explanation.** REST requires stateless requests, meaning each request must carry what it needs.

7. **Question.** Which HTTP method is normally used to retrieve data?
   **Answer.** `GET`.
   **Explanation.** In REST, `GET` corresponds to reading resources.

8. **Question.** Which technologies are used to build the user interface?
   **Answer.** HTML, CSS, and JavaScript.
   **Explanation.** These are the three classic frontend layers.

9. **Question.** Which framework is commonly used for modern frontend SPAs?
   **Answer.** React.
   **Explanation.** React is the frontend library chosen in the project and emphasized in later lectures.

10. **Question.** What is a technology stack?
    **Answer.** The combination of frontend, backend, and database technologies used in one system.
    **Explanation.** Example: React + Django + PostgreSQL is a stack.

11. **Question.** Which of the following is a relational database?
    **Answer.** MySQL.
    **Explanation.** Relational databases store structured data in related tables.

12. **Question.** Which database type is designed for flexible schema and unstructured data?
    **Answer.** NoSQL database.
    **Explanation.** NoSQL models are often chosen when the schema must stay flexible.

13. **Question.** What is the main purpose of deployment?
    **Answer.** To make the application publicly accessible.
    **Explanation.** A deployed application is reachable by users outside the local machine.

14. **Question.** Which of the following is a common PaaS deployment platform?
    **Answer.** Netlify.
    **Explanation.** Netlify is widely used for frontend hosting and static deployment.

15. **Question.** Which statement correctly describes the request-response cycle?
    **Answer.** The client sends a request and the server sends a response.
    **Explanation.** This is the basic communication loop behind web applications.

## F.3 HTML structure and semantics

1. **Question.** What does HTML stand for?
   **Answer.** HyperText Markup Language.
   **Explanation.** HTML structures content; it is not a programming language.

2. **Question.** How many tags does a normal non-empty element usually have?
   **Answer.** Two: an opening tag and a closing tag.
   **Explanation.** Example: `<p>...</p>`.

3. **Question.** What is the difference between an opening tag and a closing tag?
   **Answer.** The closing tag has a slash in front of the tag name.
   **Explanation.** Example: `<section>` versus `</section>`.

4. **Question.** What kind of tag is `<br />`?
   **Answer.** A line-break tag.
   **Explanation.** It creates a line break and is treated as an empty element.

5. **Question.** Is `<body>` an opening tag or a closing tag?
   **Answer.** Opening tag.
   **Explanation.** It begins the visible content part of the page.

6. **Question.** Is `</body>` an opening tag or a closing tag?
   **Answer.** Closing tag.
   **Explanation.** It ends the visible content part of the page.

7. **Question.** Where should the `meta` tag be placed in a valid HTML document?
   **Answer.** The `meta` tag belongs in the `head` of an HTML document.
   **Explanation.** This is a useful correction: metadata is document-level information, not something limited to one specific page type.

8. **Question.** What is the correct pattern for an image element?
   **Answer.** Use `src` and `alt`, for example `<img src="image.jpg" alt="Description">`.
   **Explanation.** The `alt` text is essential for accessibility and content meaning.

9. **Question.** What is an element without a closing tag called?
   **Answer.** An empty element.
   **Explanation.** Examples include `img` and `br`.

10. **Question.** Which of the following is an example of an empty element?
    **Answer.** `<img />` or simply `<img ...>`.
    **Explanation.** The browser treats image elements as void elements.

11. **Question.** What should attribute values be enclosed in?
    **Answer.** Quotation marks.
    **Explanation.** Example: `href="page.html"`.

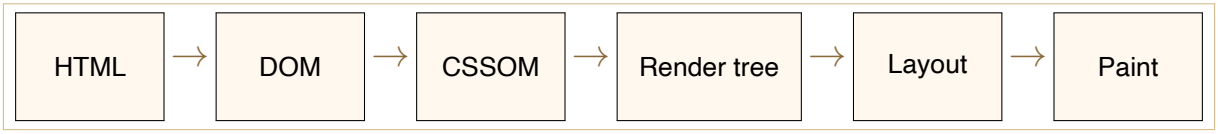
12. **Question.** Where should the files for one small website project usually be stored?
    **Answer.** In the same project folder, with a clear structure.
    **Explanation.** Small websites depend on consistent internal links and file paths.

13. **Question.** What does an anchor element such as a Google link do?
    **Answer.** It creates a clickable hyperlink to another page or resource.
    **Explanation.** Anchor tags connect pages inside a site and also to external websites.

14. **Question.** Which page is usually the welcome page of a website?
    **Answer.** The homepage.
    **Explanation.** It introduces the topic and often links to the rest of the site.

15. **Question.** What does *View Source* do?
    **Answer.** It shows the HTML source code of the page.
    **Explanation.** It is a simple way to inspect markup in the browser.

## F.4 CSS3 and responsive design



1. **Question.** How should CSS3 be described?
   **Answer.** As a collection of modular specifications, not one monolithic replacement.
   **Explanation.** CSS3 is better understood as a family of evolving modules.

2. **Question.** Which organization standardizes CSS?
   **Answer.** W3C.
   **Explanation.** W3C is responsible for the official specification process.

3. **Question.** Which stages belong to the browser rendering pipeline?
   **Answer.** DOM construction, CSSOM construction, layout, and paint/compositing.
   **Explanation.** All four stages are part of the pipeline, so they should be memorized as one connected rendering sequence.

4. **Question.** Which properties are most likely to trigger layout recalculation?
   **Answer.** `margin` and `width`.
   **Explanation.** They change element geometry and can force reflow.

5. **Question.** What is the main advantage of advanced CSS selectors?
   **Answer.** They reduce the need for extra classes and unnecessary markup.
   **Explanation.** That improves cleanliness and flexibility of stylesheets.

6. **Question.** Why is HSL often useful in design systems?
   **Answer.** It matches how humans think about color adjustments.
   **Explanation.** Hue, saturation, and lightness are easier to reason about than raw RGB values.

7. **Question.** Which statement about CSS gradients is correct?
   **Answer.** They are rendered by the browser and scale infinitely.
   **Explanation.** Gradients are vector-like visual definitions, not fixed raster images.

8. **Question.** What problem did Flexbox mainly solve?
   **Answer.** Alignment and space distribution in one dimension.
   **Explanation.** It is ideal for rows, columns, nav bars, and centered alignment.

9. **Question.** Which property defines the main direction in Flexbox?
   **Answer.** `flex-direction`.
   **Explanation.** It controls whether the layout flows in a row or column.

10. **Question.** Which three components define responsive web design in the Marcotte model?
    **Answer.** Fluid grids, flexible media, and media queries.
    **Explanation.** This is a multiple-answer concept item because all three parts are needed.

11. **Question.** What is the key difference between responsive and adaptive design?
    **Answer.** Responsive design is fluid; adaptive design relies more on prepared fixed layouts or breakpoints.
    **Explanation.** Responsive systems continuously adapt, while adaptive systems switch among chosen layouts.

12. **Question.** Which CSS units are relative by nature?
    **Answer.** `%`, `rem`, and `vw`.
    **Explanation.** `px` is absolute; the other three scale relative to context or viewport.

13. **Question.** What is the main advantage of `clamp()`?
    **Answer.** It allows fluid scaling inside fixed minimum and maximum limits.
    **Explanation.** This helps build smoother responsive typography.

14. **Question.** Which statement about CSS Grid is correct?
    **Answer.** It gives two-dimensional layout control.
    **Explanation.** Grid manages rows and columns together.

15. **Question.** Which practices improve responsive accessibility?
**Answer.** Supporting reduced motion, allowing font scaling, and adapting contrast.
**Explanation.** Relying only on pixel units works against accessibility rather than improving it.

## F.5 JavaScript fundamentals, DOM, and syntax drills

### Theory questions with complete answers

1. **Question.** What is JavaScript?
**Answer.** JavaScript is a programming language used to add behavior and interactivity to web pages.
**Explanation.** In the course sequence, HTML gives structure, CSS gives presentation, and JavaScript gives dynamic behavior.

2. **Question.** Which data types does JavaScript support in the lecture recap?
**Answer.** `undefined`, `null`, `boolean`, `string`, `symbol`, `number`, and `object`.
**Explanation.** These types are enough to understand early DOM and logic exercises.

3. **Question.** What are the main advantages of JavaScript highlighted by the course?
**Answer.** Less server interaction, immediate user feedback, increased interactivity, and richer interfaces.
**Explanation.** These strengths appear in sliders, accordions, validation, and theme toggles.

4. **Question.** What is the difference between an HTML attribute and a DOM property?
**Answer.** An attribute is the value written in the HTML markup; a property is the value exposed on the live DOM object in JavaScript.
**Explanation.** This distinction becomes visible when JavaScript updates fields or classes after page load.

5. **Question.** How can JavaScript access HTML elements?
**Answer.** With DOM selection methods such as:

```
getElementById()
getElementsByClassName()
getElementsByTagName()
querySelector()
querySelectorAll()
```

**Explanation.** A common beginner mistake is to misremember the exact class-selection method name; the correct DOM API is `getElementsByClassName`.

6. **Question.** In how many common ways can JavaScript be inserted into HTML?
**Answer.** Three ways: inline, internal, and external.
**Explanation.** External scripts are preferred in serious projects because they improve reuse and maintainability.

7. **Question.** What is the difference between `null` and `undefined`?
**Answer.** `undefined` means a value is missing or not assigned; `null` is an explicit intentional empty value.
**Explanation.** The difference matters when checking form fields, API responses, and optional data.

**Syntax and DOM test bank**

1. **Question.** Inside which HTML element do we place JavaScript?
   **Answer.** `<script>`.
   **Explanation.** That is the standard container for inline and linked script code.

2. **Question.** How do you change the content of an element with id `demo`?
   **Answer.** `document.getElementById("demo").innerHTML = "Hello World!";`
   **Explanation.** This is one of the first DOM manipulation patterns in the lecture.

3. **Question.** Where can JavaScript be inserted in a page?
   **Answer.** In the `head` or the `body`, but in practice the end of the `body` is often safest for DOM work.
   **Explanation.** This also adds the safest practical recommendation for real DOM work.

4. **Question.** What is the correct syntax for an external script called `xxx.js`?
   **Answer.** `<script src="xxx.js"></script>`.
   **Explanation.** A common incomplete answer shows only the opening tag, but the full valid HTML element needs a closing tag too.

5. **Question.** Must the external JavaScript file itself contain the `<script>` tag?
   **Answer.** No.
   **Explanation.** The external file contains only JavaScript code.

6. **Question.** How do you display `Hello World` in an alert box?
   **Answer.** `alert("Hello World");`
   **Explanation.** The built-in `alert()` function shows a browser dialog.

7. **Question.** How do you create a function named `myFunction`?
   **Answer.** `function myFunction() { }`
   **Explanation.** This is the standard declaration syntax.

8. **Question.** How do you call a function named `myFunction`?
   **Answer.** `myFunction();`
   **Explanation.** Parentheses perform the function call.

9. **Question.** How do you write an `if` statement for equality with 5?
   **Answer.** `if (i == 5) { ... }`
   **Explanation.** The lecture quiz uses ==; in modern code, === is often preferred for strict comparison.

10. **Question.** How do you check that `i` is not equal to 5?
    **Answer.** `if (i != 5) { ... }`
    **Explanation.** Again, modern code often prefers `!==` when type safety matters.

11. **Question.** How does a `while` loop start?
    **Answer.** `while (i <= 10) { ... }`
    **Explanation.** The loop condition is checked before each iteration.

12. **Question.** How does a `for` loop start in the lecture pattern?
    **Answer.** `for (i = 0; i <= 5; i++) { ... }`
    **Explanation.** It contains initialization, condition, and increment.

13. **Question.** How do you write a one-line comment in JavaScript?
    **Answer.** `// This is a comment`
    **Explanation.** Single-line comments start with double slash.

14. **Question.** How do you write a multi-line comment?
    **Answer.** `/* This comment has more than one line */`
    **Explanation.** This is the block-comment syntax.

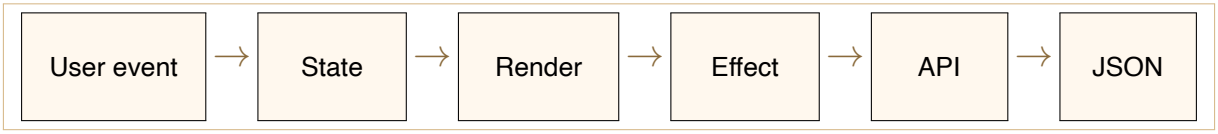
15. **Question.** What is the correct way to write an array?
    **Answer.** `var colors = ["red", "green", "blue"]`
    **Explanation.** Arrays are ordered lists stored inside square brackets.

16. **Question.** How do you round 7.25 to the nearest integer?
    **Answer.** `Math.round(7.25)`
    **Explanation.** The `Math` object provides standard numeric helpers.

17. **Question.** How do you get the higher value of `x` and `y`?
    **Answer.** `Math.max(x, y)`
    **Explanation.** This returns the larger of the provided values.

## F.6 React fundamentals mastery

The React fundamentals lecture is concept-heavy and does not expose a separate final end-of-lecture quiz block in the extracted slides in the same way as the other lectures. To keep this appendix self-sufficient, the most important React fundamentals are rewritten here as mastery checkpoints.




1. **Checkpoint.** What is React?
   **Answer.** A JavaScript library for building user interfaces through components and declarative rendering.

2. **Checkpoint.** What is a functional component?
   **Answer.** A JavaScript function that returns JSX.

3. **Checkpoint.** What are props?
   **Answer.** Read-only inputs passed from parent to child components.

4. **Checkpoint.** What is state?
   **Answer.** Internal component data that can change and trigger re-rendering.

5. **Checkpoint.** What does `useState()` provide?
   **Answer.** A state value and a setter function.

6. **Checkpoint.** Why are controlled inputs important?
   **Answer.** Because React state becomes the single source of truth for form values.

7. **Checkpoint.** How are lists rendered in React?
   **Answer.** By mapping arrays to JSX and giving each item a unique `key`.

8. **Checkpoint.** Why does React require `key` in lists?
   **Answer.** To track identity efficiently and prevent update bugs.

9. **Checkpoint.** What is conditional rendering?
   **Answer.** Displaying different UI fragments depending on state or props.

10. **Checkpoint.** What makes a SPA a SPA?
    **Answer.** One main HTML page is loaded once, and the interface changes dynamically without full page reloads.

11. **Checkpoint.** Why does React Router matter?
**Answer.** It lets a SPA have URL-based navigation and multiple visible pages without losing the SPA model.

12. **Checkpoint.** Why are JSON, Fetch, and REST already important at this stage of React learning?
**Answer.** Because React applications are data-driven and must communicate with backend APIs using structured asynchronous requests.

## F.7 React architecture, APIs, routing, and security

### 30 core questions with answers

1. **Question.** What is React primarily responsible for?
**Answer.** User interface rendering.
**Explanation.** React is not a database or authentication system; it is a UI engine.

2. **Question.** What triggers a React re-render?
**Answer.** Changing state.
**Explanation.** This is the core *UI = f(state)* idea in React architecture.

3. **Question.** Props in React are:
**Answer.** Read-only.
**Explanation.** Props flow from parent to child and should not be mutated by the receiving component.

4. **Question.** What is the correct way to update state?
**Answer.** Use the setter function.
**Explanation.** Direct assignment bypasses React's update model.

5. **Question.** What is the purpose of the `key` prop in lists?
**Answer.** Performance and identity tracking.
**Explanation.** React uses keys to compare list items between renders.

6. **Question.** What happens if you use `<a>` instead of `<Link>` in React Router?
**Answer.** A full page reload occurs and SPA state is lost.
**Explanation.** `Link` performs client-side navigation; `a` asks the browser to reload.

7. **Question.** What does `useEffect` mainly handle?
**Answer.** Side effects.
**Explanation.** Examples: API calls, subscriptions, and timers.

8. **Question.** What causes infinite loops in `useEffect`?
**Answer.** Updating a state variable that is also in the dependency array.
**Explanation.** The state change retriggers the effect repeatedly.

9. **Question.** What is a controlled input?
**Answer.** An input whose value is stored in React state.
**Explanation.** This is the standard React form pattern.

10. **Question.** Which HTTP method matches *Create* in CRUD?
**Answer.** `POST`.
**Explanation.** CRUD maps to HTTP actions in RESTful design.

11. **Question.** REST APIs are based on what style?
**Answer.** Stateless client-server architecture.
**Explanation.** That is one of REST's central constraints.

12. **Question.** What is JSON?
    **Answer.** A data exchange format.
    **Explanation.** It is text-based and widely used in web APIs.

13. **Question.** Why is JSON preferred over XML in many modern apps?
    **Answer.** It is lighter and easier to parse.
    **Explanation.** That is especially useful for JavaScript-based frontends.

14. **Question.** What does `fetch()` return?
    **Answer.** A Promise.
    **Explanation.** The response must then be awaited or handled with `then()`.

15. **Question.** Why does Fetch not throw HTTP errors automatically?
    **Answer.** Because the code must check `response.ok` manually.
    **Explanation.** A resolved fetch can still represent a 404 or 500 response.

16. **Question.** What advantage can Axios have over Fetch in some projects?
    **Answer.** Automatic JSON parsing and interceptors.
    **Explanation.** Axios centralizes some common request behavior.

17. **Question.** Which three UI states are recommended during data fetching?
    **Answer.** Loading, error, and success.
    **Explanation.** This is one of the strongest recurring lessons in API-driven UI design.

18. **Question.** What is JWT?
    **Answer.** A signed authentication token.
    **Explanation.** It proves identity or authorization when sent correctly.

19. **Question.** Is JWT encrypted by default?
    **Answer.** No; it is usually signed, not encrypted.
    **Explanation.** That is why sensitive information should not be stored inside it.

20. **Question.** How is JWT usually sent to the backend?
    **Answer.** In the `Authorization` header.
    **Explanation.** This is the typical bearer-token pattern.

21. **Question.** What does `Bearer` mean in that header?
    **Answer.** Whoever possesses the token can present it for access.
    **Explanation.** That is why the token must be protected carefully.

22. **Question.** What is a Single Page Application?
    **Answer.** A web app that loads one main HTML page and updates content dynamically.
    **Explanation.** Navigation and rendering happen inside the client without full reload.

23. **Question.** What is prop drilling?
    **Answer.** Passing props through many levels only to reach a deep child component.
    **Explanation.** It can make component trees harder to maintain.

24. **Question.** What is the role of `BrowserRouter`?
    **Answer.** It enables URL-based routing in a React SPA.
    **Explanation.** It is the routing container that works with routes and links.

25. **Question.** Which statement about state is true?
    **Answer.** State is internal to the component.
    **Explanation.** It is not automatically global and it does not replace props.

26. **Question.** What is REST mainly focused on?
    **Answer.** Resources and HTTP methods.
    **Explanation.** That is the basis of endpoint and CRUD design.

27. **Question.** What does GraphQL allow that REST does not emphasize in the same way?
    **Answer.** Requesting exactly the data needed.
    **Explanation.** This helps reduce over-fetching and under-fetching.

28. **Question.** Why should sensitive data not be stored in JWT?
    **Answer.** Because JWT payloads can be decoded.
    **Explanation.** Signed does not mean secret.

29. **Question.** What is the correct UI behavior when an API request fails?
    **Answer.** Show an error state.
    **Explanation.** Silent failure or blank screens are poor UX.

30. **Question.** What is the main architectural benefit of a separate API layer?
    **Answer.** Centralized communication logic.
    **Explanation.** This improves maintainability, consistency, and testing.

### 4 analytical questions with model answers

1. **Question.** A component fetches data in `useEffect` but creates an infinite loop. Explain why and how to fix it.
   **Model answer.** The infinite loop happens when the effect updates a state value and that same state value appears in the dependency array. The state change causes a re-render, React detects a dependency change, and the effect runs again. If the effect should run only once on mount, use an empty dependency array. If it depends on a search term or id, then depend only on that true external input.
   **Explanation.** This is one of the most important analytical patterns in React effect management.

2. **Question.** Compare REST and GraphQL for a large e-commerce application. Which would you choose and why?
   **Model answer.** REST is resource-based, easier to cache with standard HTTP tools, and usually simpler to scale operationally. GraphQL gives flexible queries and can reduce over-fetching in complex UIs. For a large e-commerce system with heavy traffic and strong caching needs, REST is often the safer default; GraphQL becomes attractive when the client requires highly variable nested data.
   **Explanation.** This answer emphasizes practical trade-offs rather than memorizing slogans.

3. **Question.** Why does React use declarative rendering instead of imperative DOM manipulation?
   **Model answer.** Declarative rendering lets the developer describe the UI as a function of state. React then computes the necessary DOM updates. This reduces manual inconsistency, improves readability, and scales better because the UI stays synchronized with application state.
   **Explanation.** This is a core architectural reason for using React.

4. **Question.** Why does separating API logic from UI components improve scalability?
   **Model answer.** It prevents fetch logic from being scattered across many components, centralizes error handling and token logic, and lets UI code focus on rendering. It also makes testing easier and allows several developers to work on the API layer and UI layer with less coupling.
   **Explanation.** This is a strong analytical answer because it connects architecture, maintainability, and team workflow.

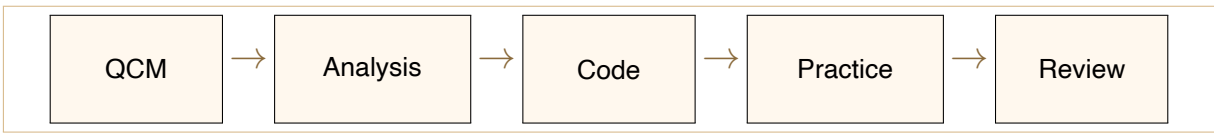
### F.8 Practice map

| Practice area | Where the appendix now covers it |
|---|---|
| Environment setup and project structure | frontend/backend learning paths, project construction order, defense preparation |
| Semantic HTML pages and forms | frontend exercises 1 to 4, practical tasks 31 and 32, HTML study bank |
| CSS layout and responsive design | frontend exercises 2 to 4, practical tasks 33 to 36, complete CSS study bank |
| DOM, events, validation, and browser storage | frontend exercises 5 to 15, practical tasks 37 to 40, complete JavaScript bank |
| React fundamentals | frontend learning path, worked React exercises, React mastery section |
| Routing, APIs, and loading/error states | frontend learning path, CC1 practice, API-state explanations, advanced React bank |

Double-check conclusion: this appendix now stands on its own as a revision book. It covers web architecture, HTML, CSS, JavaScript, React, API integration, backend thinking, and hands-on practice in one place, with worked answers and explanations throughout.

## G CC1 Exam Practice and Answer Key

This section is built to match the announced **CC1** style: theory questions across web architecture, HTML, CSS, JavaScript, React, and API thinking, plus practical exercises in **HTML, CSS, and JavaScript**. It is intentionally placed at the end of the appendix as a final training section after the broader theory review and the full independent study bank. The answers are written in English, but the same reasoning can be reproduced in French during the exam.




### G.1 Topic alignment

| Topic block | Main ideas reused in the exercises |
|---|---|
| Web architecture and deployment | client/server roles, HTTP, HTTPS, REST, stack, deployment, environment setup |
| HTML and accessibility | semantic HTML, heading hierarchy, links, images, forms, accessibility basics |
| CSS and responsive layout | box model, Flexbox, Grid, media queries, responsive design, typography, spacing |
| JavaScript and DOM logic | DOM, query selectors, events, `preventDefault`, localStorage, FAQ, theme toggle, validation |
| React fundamentals | components, props, state, `useState`, controlled input, lists, filtering, SPA thinking |
| React APIs and routing | `useEffect`, loading/error/success states, API layer, routing, protected flows, infinite loop traps |

## G.2 Part A: 10 multiple-choice questions

1. **Question.** What is the primary role of the client in a web application?
   **Options.** A. Manage the database. B. Display the interface and handle user interaction. C. Execute server business logic. D. Store backend credentials.
   **Correct answer.** B.
   **Explanation.** The client is responsible for presentation and interaction, while the server handles business logic and database operations.

2. **Question.** Which of the following is **not** compatible with the stateless REST model taught in the course?
   **Options.** A. Resource-based design. B. Stateless communication. C. Uniform interface. D. Server keeps hidden per-user session state between requests.
   **Correct answer.** D.
   **Explanation.** In a stateless REST design, each request must carry the information needed to process it.

3. **Question.** Why are semantic HTML tags preferred over many generic `div` blocks?
   **Options.** A. They reduce CSS file size automatically. B. They improve meaning, accessibility, and maintainability. C. They replace JavaScript. D. They remove the need for headings.
   **Correct answer.** B.
   **Explanation.** Semantic HTML improves accessibility, SEO, and code readability at the same time.

4. **Question.** What problem did Flexbox mainly solve?
   **Options.** A. Two-dimensional page layout. B. Database modeling. C. One-dimensional alignment and space distribution. D. Browser routing.
   **Correct answer.** C.
   **Explanation.** Flexbox is a one-dimensional layout system, ideal for navigation bars and alignment.

5. **Question.** Which statement about CSS Grid is correct?
   **Options.** A. It only works for text. B. It is limited to one dimension. C. It gives two-dimensional layout control. D. It replaces HTML structure.
   **Correct answer.** C.
   **Explanation.** Grid is the right tool when the layout needs rows plus columns, dashboards, or content-and-sidebar pages.

6. **Question.** What does `document.querySelectorAll(".card")` return?
   **Options.** A. One element. B. A NodeList of matching elements. C. A CSS rule. D. A string.
   **Correct answer.** B.
   **Explanation.** `querySelectorAll()` returns a list-like collection of matching elements that can be looped over.

7. **Question.** Why is `event.preventDefault()` often used in form handling?
   **Options.** A. To clear localStorage. B. To stop React rendering. C. To prevent the browser's default submission reload. D. To create an API route.
   **Correct answer.** C.
   **Explanation.** This prevents the browser's default page reload so JavaScript or React can validate and control the submission flow.

8. **Question.** In React, props are best described as:
   **Options.** A. Mutable global variables. B. Read-only values passed from parent to child. C. Database rows. D. CSS classes.
   **Correct answer.** B.
   **Explanation.** Props move from parent to child and should not be mutated directly by the receiving component.

9. **Question.** What usually triggers a React re-render?
   **Options.** A. Changing a regular local variable. B. Changing component state. C. Writing a CSS rule. D. Opening DevTools.
   **Correct answer.** B.
   **Explanation.** React works with the idea *UI = f(state)*. When state changes, React computes the new UI.

10. **Question.** What is the main purpose of `useEffect()`?
    **Options.** A. Styling JSX. B. Declaring routes. C. Handling side effects such as data fetching. D. Creating HTML structure.
    **Correct answer.** C.
    **Explanation.** `useEffect` is the standard hook for side effects such as API calls, subscriptions, and timers.

## G.3 Part B: 10 analytical questions and model answers

1. **Question.** Why does semantic HTML improve accessibility and SEO at the same time?
   **Model answer.** Semantic tags such as `header`, `nav`, `main`, and `section` describe the role of content instead of only its visual placement. Screen readers can navigate the structure more intelligently, and search engines can better understand which parts are titles, navigation, or content blocks.
   **Why this matters.** Semantic tags are not decoration; they are the foundation of good structure.

2. **Question.** When should you choose Flexbox, and when should you choose Grid?
   **Model answer.** Choose Flexbox when the problem is one-dimensional, for example a navigation bar, a row of buttons, or centered content. Choose Grid when the layout needs rows and columns together, such as a dashboard, card layout, or content-plus-sidebar page.
   **Why this matters.** Confusing Flexbox and Grid often leads to harder CSS and weaker layouts.

3. **Question.** Explain the difference between responsive and adaptive design.
   **Model answer.** Responsive design is fluid and adjusts continuously using flexible layouts, relative units, and media queries. Adaptive design relies more on fixed breakpoints and prepared layouts for selected screen sizes.
   **Why this matters.** Good CSS work requires design reasoning, not only syntax memorization.

4. **Question.** Why can changing `width` or `margin` be more expensive than changing `color`?
   **Model answer.** Changes to width or margin can force layout recalculation, because the browser may need to recompute positions and sizes of elements. A color change usually affects painting only, so it does not rebuild layout.
   **Why this matters.** This explains why some CSS changes are visually cheap while others trigger expensive layout recalculation.

5. **Question.** Why is JavaScript often placed at the end of the `body` or executed after DOM readiness?
   **Model answer.** If JavaScript runs before the DOM exists, selectors may fail because the elements are not yet available. Loading the script after the HTML or waiting for DOM readiness ensures that DOM operations work reliably.
   **Why this matters.** DOM timing bugs are common, so scripts must run only after the elements exist.

6. **Question.** Why do we use `preventDefault()` in client-side validation?
   **Model answer.** By default, a form submission reloads the page. `preventDefault()` keeps the browser on the page so JavaScript can validate fields, show errors, and only submit when

the data is acceptable.
**Why this matters.** Validation messages must stay visible long enough for the user to understand the problem.

7. **Question.** Why are keys required when rendering lists in React?
**Model answer.** Keys give each rendered item a stable identity. This helps React compare the old list with the new list efficiently and avoid UI bugs when items are added, removed, or reordered.
**Why this matters.** Keys improve both rendering correctness and performance, not just warning removal.

8. **Question.** Explain why React prefers declarative rendering over manual DOM manipulation.
**Model answer.** In declarative rendering, the developer describes the UI that should exist for a given state. React then computes the necessary DOM updates. This reduces inconsistency, improves readability, and scales better than manually editing DOM nodes step by step.
**Why this matters.** This is one of the main architectural reasons React scales better than manual DOM manipulation.

9. **Question.** Why can `useEffect` create an infinite loop?
**Model answer.** An infinite loop appears when the effect updates a state variable and that same variable is also in the dependency array. The update triggers a re-render, the dependency changes, the effect runs again, and the cycle repeats.
**Why this matters.** This is a classic analytical exam pattern, so the mechanism should be explained clearly and precisely.

10. **Question.** Why is the `loading/error/success` pattern considered professional UI behavior?
**Model answer.** Network requests are asynchronous and can succeed, fail, or still be in progress. A professional UI makes all three situations visible so the user always understands what is happening instead of seeing a blank or confusing screen.
**Why this matters.** These three states are a serious frontend practice because they make asynchronous behavior understandable to the user.

## G.4 Part C: 10 code analysis and code completion exercises

1. **Code analysis.** Identify the structural error and correct it.

```
<main>
  <h1>EventHub</h1>
  <h3>Upcoming events</h3>
  <h2>Spring Hackathon</h2>
</main>
```

**Answer.** The heading hierarchy is incorrect because `h3` appears before `h2`. The corrected structure is:

```
<main>
  <h1>EventHub</h1>
  <h2>Upcoming events</h2>
  <h3>Spring Hackathon</h3>
</main>
```

**Explanation.** Logical heading order matters because headings express document structure.

2. **Code completion.** Complete the image element so it is accessible.

```
<img src="conference.jpg" ________>
```

**Answer.**

```
<img src="conference.jpg" alt="Students attending a conference talk">
```

**Explanation.** The `alt` attribute is mandatory because images need accessible meaning.

3. **Code analysis.** What is the total outer width of the box?

```
.card {
  width: 200px;
  padding: 20px;
  border: 2px solid #000;
  margin: 10px;
}
```

**Answer.** With the default content-box model, the outer width is `200 + 40 + 4 + 20 = 264px`.
**Graphical result.**

```
margin | border | padding | content | padding | border | margin
 10px  |  2px   |  20px   | 200px   |  20px   |  2px   | 10px
```

**Explanation.** This type of calculation helps students understand spacing bugs and the box model clearly.

4. **Code completion.** Complete the media query so the menu becomes vertical on small screens.

```
nav ul {
  display: flex;
  flex-direction: row;
}

@media (max-width: 768px) {
  nav ul {
    ______________________;
  }
}
```

**Answer.**

```
flex-direction: column;
```

**Result.**

```
Desktop: Home | About | Contact
Mobile:
Home
About
Contact
```

**Explanation.** This is one of the most common practical CSS patterns for responsive navigation.

5. **Code analysis.** What is logged in the console?

```
const items = document.querySelectorAll(".card");
console.log(items.length);
```

**Answer.** The console logs the number of elements in the page with class `card`.
**Explanation.** `querySelectorAll()` returns a NodeList, not a single element.

6. **Code completion.** Finish the click handler so it toggles the answer.

```
const button = document.querySelector(".faq-question");
const answer = document.querySelector(".faq-answer");
button.addEventListener("click", () => {
  answer.classList._______________("open");
});
```

**Answer.**

```
toggle
```

**Result.**

```
click 1 -> answer visible
click 2 -> answer hidden
```

**Explanation.** This is the core FAQ open-close pattern.

7. **Code completion.** Fix the form submit handler.

```
form.addEventListener("submit", (event) => {
  _______________________;
  console.log("validate fields");
});
```

**Answer.**

```
event.preventDefault()
```

**Explanation.** Without this line, the page reloads immediately and the validation UI becomes unreliable.

8. **Code completion.** Complete the localStorage logic for theme persistence.

```
localStorage.setItem("theme", "dark");
const savedTheme = ___________________________;
```

**Answer.**

```
localStorage.getItem("theme")
```

**Result.**

```
savedTheme === "dark"
```

**Explanation.** This is the standard pattern for saving theme or language preferences in the browser.

9. **Code analysis.** Why is this React update incorrect?

```
tasks.push(newTask)
setTasks(tasks)
```

**Answer.** It mutates the existing array instead of creating a new one. The correct pattern is:

```
setTasks([...tasks, newTask])
```

**Explanation.** Direct mutation can cause stale or confusing UI behavior because React expects immutable state updates.

10. **Code analysis.** Explain the bug and fix the dependency array.

```
useEffect(() => {
  fetchTasks().then(setTasks)
}, [tasks])
```

**Answer.** The effect depends on `tasks` while also updating `tasks`, which can cause an infinite loop. If the fetch should run only on mount, use:

```
useEffect(() => {
  fetchTasks().then(setTasks)
}, [])
```

**Explanation.** This is the standard fix when an effect should run only once on component mount.

## G.5 Part D: 10 worked practical HTML, CSS, and JavaScript exercises

**Important.** The 10 practical tasks below are also fully corrected. The code block is the exact answer you can type in a practice file, the terminal block shows the visual or logical result, and the explanation tells you what to memorize for the exam.

1. **Practical task 31.** Build the semantic skeleton of a correct main page.

```
<body>
  <header>
    <h1>EventHub</h1>
    <nav>
      <a href="events.html">Events</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>
  <main>
    <section>
      <h2>Upcoming events</h2>
      <p>Discover the next academic activities.</p>
    </section>
  </main>
  <footer>Author: EventHub team</footer>
</body>
```

**Expected visual result.**

```
+-------------------------------------+
| EventHub   Events   Contact         |
| Upcoming events                     |
| Discover the next academic activities|
| Author: EventHub team               |
+-------------------------------------+
```

**Explanation.** This is the correct semantic base before any CSS is added.

2. **Practical task 32.** Create an accessible login form with labels.

```html
<form>
  <label for="email">Email</label>
  <input id="email" type="email" />

  <label for="password">Password</label>
  <input id="password" type="password" />

  <button type="submit">Login</button>
</form>
```

**Expected visual result.**

```
Email
[_______________]
Password
[_______________]
[ Login ]
```

**Explanation.** Correct form structure matters even before functionality is added.

3. **Practical task 33.** Style a navigation bar with Flexbox.

```css
nav ul {
  display: flex;
  justify-content: space-between;
  align-items: center;
  list-style: none;
  padding: 0;
}
```

**Expected visual result.**

```
Home                About               Contact
```

**Explanation.** Flexbox is ideal for horizontal alignment and navigation layout.

4. **Practical task 34.** Create a two-column page layout with Grid.

```css
.main-layout {
  display: grid;
  grid-template-columns: 3fr 1fr;
  gap: 20px;
}
```

**Expected visual result.**

```
+---------------------------+-----------+
| Main content              | Sidebar   |
+---------------------------+-----------+
```

**Explanation.** This is a classic content-plus-sidebar pattern built with Grid.

5. **Practical task 35.** Show the visual difference between padding and margin.

```
.box {
  margin: 20px;
  padding: 16px;
  border: 2px solid #182620;
}
```

**Graphical result.**

```
outside gap = margin
+----------------------+
|  inside gap = padding|
|  content             |
+----------------------+
```

**Explanation.** This helps students reason visually about spacing instead of only memorizing definitions.

6. **Practical task 36.** Add a responsive rule for mobile screens.

```
@media (max-width: 768px) {
  .main-layout {
    grid-template-columns: 1fr;
  }
}
```

**Expected result.**

```
Desktop:
[Main content][Sidebar]

Mobile:
[Main content]
[Sidebar]
```

**Explanation.** This is one of the most realistic CC1 CSS tasks because it combines Grid and media queries.

7. **Practical task 37.** Change a title using DOM selection and a click event.

```
const title = document.querySelector("#hero-title");
const button = document.querySelector("#change-title");

button.addEventListener("click", () => {
  title.textContent = "Registration Open";
});
```

**Expected result.**

```
Before click: Welcome to EventHub
After click:  Registration Open
```

**Explanation.** Direct DOM interaction like this is the basis of more complex widgets.

8. **Practical task 38.** Implement a FAQ accordion with show/hide logic.

```javascript
const button = document.querySelector(".faq-question");
const answer = document.querySelector(".faq-answer");

button.addEventListener("click", () => {
  answer.classList.toggle("open");
});
```

**Expected visual result.**

```
Question: What is EventHub?
click -> answer opens
click again -> answer closes
```

**Explanation.** This is a standard FAQ accordion pattern.

9. **Practical task 39.** Validate a contact form without reloading the page.

```javascript
const form = document.querySelector("form");
const message = document.querySelector("#message");
const errorBox = document.querySelector("#message-error");
const successBox = document.querySelector("#form-success");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  if (message.value.length < 10) {
    errorBox.textContent = "Message must contain at least 10 characters."
   ;
    return;
  }

  successBox.textContent = "Sent!";
});
```

**Expected result.**

```
short message -> "Message must contain at least 10 characters."
valid message -> "Sent!"
```

**Explanation.** Good validation must show visible error messages and a clear success state.

10. **Practical task 40.** Implement a light/dark theme toggle that persists after reload.

```javascript
const themeButton = document.querySelector("#theme-toggle");

themeButton.addEventListener("click", () => {
  document.body.classList.toggle("dark");
  const nextTheme = document.body.classList.contains("dark")
    ? "dark"
    : "light";
```

```
    localStorage.setItem("theme", nextTheme);
  });
```

**Expected design result.**

```
click -> body background becomes dark
reload page -> saved theme is restored
```

**Explanation.** This combines three important browser ideas at once: CSS classes, events, and localStorage persistence.

## G.6 How to revise effectively

1. Start with Part A and answer the 10 QCM without looking at the solutions.
2. Move to Part B and try to write each answer in 3 to 5 sentences.
3. For Part C, rewrite the corrected code by hand so the patterns become automatic.
4. For Part D, reproduce the practical solutions in a clean folder and test them in the browser.
5. Revise JavaScript before React hooks, because React hooks depend on solid JavaScript reasoning.
6. Memorize the three frontend layers as a chain: HTML gives structure, CSS gives layout and appearance, JavaScript gives behavior.
7. Memorize the React chain as well: props bring data in, state stores changing values, effects connect the UI to asynchronous work.
8. Memorize the API chain as one flow: client request, server response, JSON payload, and UI loading/error/success states.

The appendix is meant to be studied repeatedly. Frontend learning should begin with HTML, CSS, JavaScript, and React in that order. Backend learning should begin with API thinking, then Django models, serializers, routes, and only after that the Node.js and Express comparison. The complete independent study bank should be used first, and the CC1 practice section should be used last as the final exam-training block.